Cricket Shot Recognition: Comparing Gradient Features and a Frozen CNN

Hrushikesh Roop Avvari (230249) Voora Rakesh (231174)
Gokavarapu Manikanta Sri Akshith (230418)
Lokesh Kumar Moond (230606) Bollu Sharath chandra (230303)
Indian Institute of Technology Kanpur
EE604, October 2026

Abstract

We study a small, practical task: classify one cricket image as drive, leg glance/flick, pull or sweep. We compare a hand-crafted histogram of oriented gradients (HOG) with frozen MobileNetV2 features followed by a linear classifier. Our implemented extension combines global and fixed regional CNN features with their absolute difference, and exposes disagreement between separate view classifiers. On an unchanged test set of 945 images in 70 visual groups, HOG obtains 35.3% accuracy and 34.5% macro F1; the global CNN obtains 63.0% and 62.9%. Contrast obtains 65.5% and 64.9%, only three additional correct predictions over concatenation. Group-based uncertainty does not establish a clear advantage. A duplicate audit substantially reduces the optimistic preliminary result, and temperature scaling worsens test calibration. We report these limitations alongside a local demo and reproducible predictions. The study is exploratory because grouping was revised after earlier evaluation.

1. Introduction

A collection of labeled cricket images can help retrieve examples of batting shots. Our task takes one image containing a shot and returns one of four supplied labels. It assumes that someone has already chosen the image. It does not locate the batter, identify a contact frame or grade batting technique.

Visual differences can be subtle. The bat may be blurred, a frame may show preparation rather than contact, and camera angle or background may dominate. We ask two focused questions. How does a gradient descriptor compare with pretrained CNN features on the same images? Does comparing a global view with a fixed central region provide evidence beyond simply concatenating them?

Our foundation is feature extraction followed by classification. A single global descriptor mixes the batter with the surrounding scene; a fixed crop offers a second view but may discard useful context. We test whether an explicit difference between the two representations helps a linear classifier use them together. We also implement a small learned gate to ask whether image-dependent weighting helps more than fixed fusion. Our contribution is this controlled investigation, including a HOG baseline and an inspectable agreement diagnostic. The components are established; the experiments test their integration rather than claiming a new backbone or an unprecedented combination.

2. Related work and positioning

HOG accumulates gradient orientations into normalized spatial blocks [3]. Our documented variant uses this idea for whole-image classification; it does not reproduce the original pedestrian detector. MobileNetV2 is an established efficient CNN [10]. We select public ImageNet ONNX weights [9] and freeze them, following Lecture 15's use of a pretrained feature extractor with an ML classifier. Freezing keeps the cricket-specific training to a small classification head.

Earlier cricket studies learn from video sequences. Sen et al. [11] combine CNN frame features with a GRU and study frozen and partly fine-tuned VGG16 variants on ten shot classes. Kang's preprint [8] compares sequence models and proposes EfficientNet-B0 with a bidirectional GRU and temporal attention. Table 1 states how our narrower image task differs. Their data, labels and splits differ from ours, so published percentages would not be controlled baselines.

KUCricShot [6] supplies a separate four-shot dataset with 1,278 examples according to its authors' repository; we did not use it. Temperature scaling [5] and selective classification [4] are established. Our threshold heuristic provides no guaranteed risk bound.

Study	Input and method	Difference here
Sen et al. [11]	Ten-class video; CNN features and GRU; frozen and partly fine-tuned VGG16 variants.	One image, four labels; frozen MobileNetV2 with linear heads and two spatial views.
Kang [8]	Ten-class video; proposed EfficientNet-B0, bidirectional GRU and temporal attention.	No sequence model; explicit view-difference ablation against concatenation.

Table 1. Prior cricket methods and concrete scope differences. Different datasets and protocols prevent direct score comparison.

Label	Train	Val.	Test
Drive	734	244	245
Glance/flick	672	224	224
Pull	756	252	252
Sweep	672	224	224
Total	2834	944	945

Table 2. Retained image counts; all visual groups remain in one split.

3. Data and evaluation

We use version 2 of Aneesh's *Cricket Shots Dataset* on Kaggle [2]. Its 4,724 files have four upstream labels. Drive and leg glance/flick each combine related stroke types. We retain these labels and do not claim original collection or annotation. The downloaded metadata reports an unknown license. Provenance and checksums are recorded. Source control excludes the raw archive; five selected JPEG illustrations already used in the slides also provide demo inputs.

Decoded RGB hashes remove one excess exact copy. A visual audit connects qualifying near copies, including rotated and mirrored versions, then keeps each connected group in one split. It leaves 4,723 representatives in 363 visual groups. The initial hash-only contrast accuracy was 96.3%. Visual review found missed transformed pictures, so grouping was strengthened and the comparison rerun. The corrected result is 65.5%. Both predictions and both grouping records remain available. This history makes the final study exploratory; the new HOG comparison uses the corrected split unchanged.

Training has 2,834 images in 221 groups, validation 944 in 72 groups, and test 945 in 70 groups. These are fixed stratified group folds with seed 604: fold 0 is test, fold 1 validation, and the others training. This is one holdout, not five-fold cross-validation. The source has no verified match or player identities. Visual groups are not necessarily independent photographs or matches. Appendix A.1 gives detailed audit criteria.

4. Proposed method

4.1. Common geometry

We apply EXIF orientation, convert to RGB and bound the image to a 1,024-pixel box. The CNN uses the image extent and a central crop with $x \in [0.15W, 0.85W]$, $y \in [0.05H, 0.95H]$. The crop is a simple prior that the batter is near the center, reducing some background without training a detector. Each view is fitted to 224 by 224 while preserving aspect ratio and cropping the center. Even the global branch loses peripheral pixels at this step. The crop can miss an off-center batter or remove the bat and scene context; keeping a global branch partly addresses this risk, without guaranteeing localization.

4.2. HOG baseline

HOG uses the identical global geometry, converted to 8-bit luminance and scaled to $[0, 1]$. Central differences give g_x, g_y, magnitude $m = \sqrt{g_x^2 + g_y^2}$ and unsigned angle $\theta = \operatorname{atan2}(g_y, g_x) \bmod 180°$. Boundary derivatives are zero. Each 16 by 16 cell accumulates *magnitudes*, not pixel counts, into nine bins centered at $0, 20, \ldots, 160°$. A pixel splits its vote linearly between adjacent orientation bins, wrapping at $180°$. Votes do not interpolate across cell boundaries.

A 2 by 2 cell block has 36 values and moves one cell per step. Each block is divided by $\sqrt{\sum_i h_i^2 + 10^{-10}}$. The 14 by 14 cells yield 13 by 13 blocks: $13 \cdot 13 \cdot 36 = 6,084$ concatenated values. Final per-image L2 normalization gives the classifier input. There is no learned scaling, PCA or feature selection. These fixed settings are not claimed to be optimal HOG settings. The lecture's signed-angle example and our unsigned bins are distinct variants of the same principle.

4.3. Frozen CNN and contrast

We choose frozen MobileNetV2 to keep feature extraction and the local demo practical on a CPU. With only 221 training visual groups, fitting a small head also limits the number of cricket-specific parameters. This is a design rationale, not a comparison proving that freezing or this backbone is optimal. The CNN uses ImageNet RGB mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$. We remove the ImageNet classifier and retain the pooled 1,280-value output. No CNN parameters are updated on cricket images. Per-view L2 normalization prevents raw embedding magnitude from setting the relative scale of the global and crop blocks; it does not remove background or prove equal usefulness. No normalization ablation was conducted. For these normalized vectors

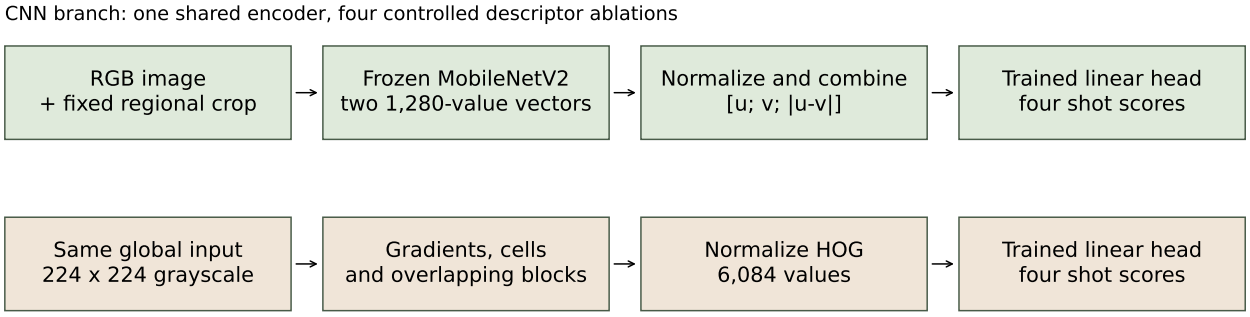


Figure 1. Two feature approaches on the same split. Only the linear cricket classifiers learn from training images. CNN ablations share a frozen encoder. The fixed crop is a regional view, not a detected or segmented batter.

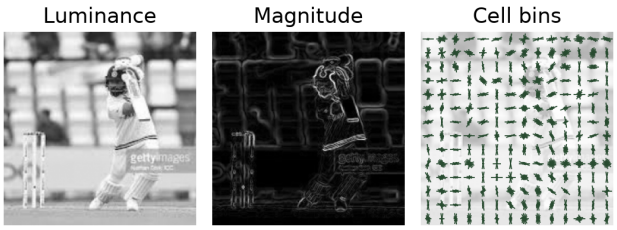


Figure 2. A selected test image traced through luminance, gradient magnitude and magnitude-weighted cell bins. This explains HOG; it is not a localization result or a representative performance sample.

Feature	Dim.	C	Acc.	F1
HOG	6084	0.1	35.3	34.5
CNN global	1280	10	63.0	62.9
CNN crop	1280	10	62.4	61.7
Concat.	2560	10	65.2	64.6
Contrast	3840	10	65.5	64.9

Table 3. All-test accuracy and macro F1 in percent. Same images, labels, groups, split and regularization grid; HOG has no ImageNet pretraining.

u, v, compare u, v, $\mathrm{Norm}([u; v])$, and

$$z = \mathrm{Norm}([u; v; |u - v|]) \in \mathbb{R}^{3840}. \quad (1)$$

Absolute difference gives a linear classifier a nonlinear relationship between the views. A signed difference would already be expressible by a linear map of concatenation. This motivates the construction without proving it improves recognition. Coordinates have no claimed physical interpretation, and the crop vector is not a local keypoint descriptor.

All five representations use class-balanced, L2-regularized multinomial logistic regression. Logits are $a = Wz + b$ and scores are $p_c = \exp(a_c/T)/\sum_j \exp(a_j/T)$. Training minimizes weighted multiclass cross-entropy with regularization. Contrast stores a 4×3840 matrix and four biases: 15,364 parameters. These cricket parameters are learned; the CNN and HOG extraction rules stay fixed.

5. Experiments

5.1. Controlled comparison

Every head uses the corrected manifest and $C \in \{0.1, 1, 10, 100\}$. Validation macro F1 selects C, with smaller values winning ties. HOG's configuration and selection are saved before test scoring. No new split, seed, feature setting or threshold was searched for a better test result. We retain the original CNN heads and predictions. HOG is an exploratory extension after prior results on this archive were known.

The original validation set also selected CNN temperatures and the review threshold. This reuse can overfit validation. We disclose it and preserve the choices; separate calibration data is future work. HOG uses $T = 1$ and does not alter the review gate.

HOG selects $C = 0.1$ and reaches 49.4% validation macro F1, falling to 34.5% on test. Its 35.3% accuracy is below all four CNN configurations. This supports pretrained features *for this configuration and split*. It does not isolate color, pretraining or architecture effects, or establish universal HOG inferiority.

Contrast gets 619/945 correct, versus 616/945 for concatenation: only +0.32 accuracy points. A paired group-bootstrap interval is -1.46 to +2.38 points, spanning zero. Concatenation has the highest validation macro F1. We retain the pre-existing contrast deployment, but do not present a substantial or reliable advantage.

Comparing the fixed test decisions shows 17 concatena-

View heads	Images	Concat.	Contrast	Gain	Loss
Agree	713	499	499	3	3
Disagree	232	117	120	14	11
All	945	616	619	17	14

Table 4. Correct counts conditioned on global/crop agreement. Gain/loss compares contrast with concatenation; post-hoc descriptive evidence only.

Feature	Acc. 95%	F1 95%
HOG	25.9–48.1	24.9–44.6
CNN global	49.5–74.2	44.6–72.0
CNN crop	45.3–76.7	42.1–72.3
Concat.	48.9–77.9	43.7–74.9
Contrast	49.9–77.6	45.8–74.2

Table 5. Conditional visual-group bootstrap intervals, in percent, from saved individual predictions.

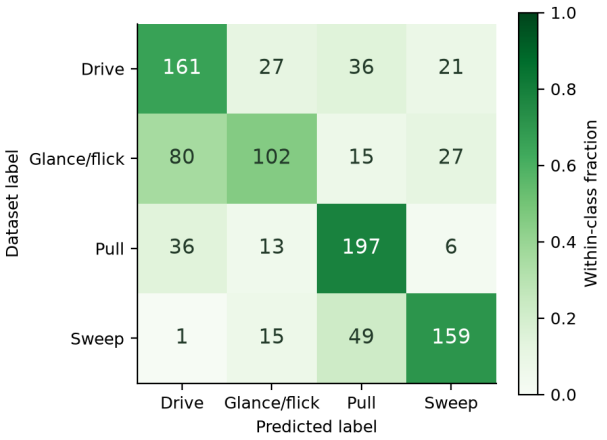


Figure 3. Contrast confusion counts on all test images. Rows are dataset labels; color is within-class fraction. Review does not remove errors from this matrix.

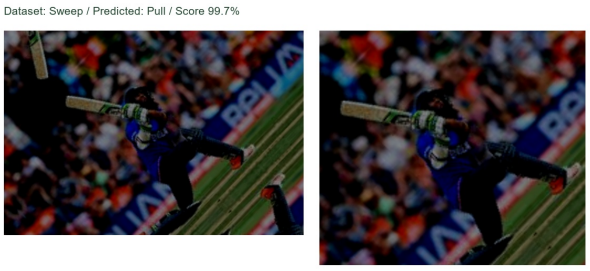


Figure 4. Dataset label: sweep; prediction: pull (99.7% score). This error was selected after evaluation and retained in the demo, alongside successes and disagreement.

tion errors corrected by contrast, but 14 correct concatenation decisions lost. Among 232 images where the separate global and crop heads disagree, contrast gets 120 correct versus 117 for concatenation. Among 713 where they agree, each gets 499 correct. These post-hoc subsets describe decisions, not a causal explanation or an independent validation of the feature. For example, dataset-labeled drive `image_00696` changes from concatenation's pull to contrast's drive; `image_00017`, also labeled drive, changes in the opposite direction, from correct drive to incorrect glance/flick. These are the first IDs in the gain/loss categories, selected after evaluation. Both directions matter when assessing the motivation.

Intervals resample the 70 visual groups with replacement in 2,000 draws, seed 17, retaining each drawn group's images. Identical draws give paired comparisons. Statistics remain image-weighted within each resample. These conditional percentile intervals omit training and split variability and cannot repair unknown match correlation or missed copies. They do not treat 945 images as independent original photographs.

5.2. Errors and model scores

Glance/flick recall is 45.5% on 224 examples, while pull recall is 78.2% on 252. HOG predicts pull for 110 of 224 sweep images. Similar stances, fixed spatial cells and camera layout are plausible limitations, not verified causes for every error.

Temperature scaling retains the top label but worsens test ECE from 0.043 to 0.094 and NLL from 0.859 to 0.898 (ten equal-width ECE bins). It did not demonstrate improved calibration. A softmax score is relative to the supported labels, not a guarantee that an arbitrary image is correct or belongs to these classes.

The unchanged secondary review rule requires score at least 0.70 and agreement of global, crop and contrast heads. It accepts 557/945 images, with 433 correct: 58.9% coverage and 77.7% accepted accuracy. Confidence alone gives 62.2% coverage and 77.6% accepted accuracy. Thus the observed accuracy benefit is negligible while more images request review. We retain agreement as a visible diagnostic of conflicting views; it reuses the same encoder outputs. The 90% validation target is not a test result or guarantee. Correlated heads can agree on a wrong label. Appendix A.2 retains the detailed diagnostics.

5.3. Learned weighting follow-up

The small contrast gain motivates a further question: can the model learn when to trust each view? We implement a compact mixture of experts, an established idea [7]. For

Score fusion	Val. F1	Test acc.	Test F1
Uniform weights	78.6	65.1	64.5
Learned constant	78.8	65.4	64.8
Learned adaptive	78.8	65.4	64.8

Table 6. Exploratory fusion follow-up, percent. Identical expert heads and split; unscaled expert scores. Adaptation does not beat the constant-weight control.

global, crop and contrast score vectors p_j, a gate predicts

$$\alpha = \mathrm{softmax}(B^T s + b), \qquad q = \sum_{j=1}^{3} \alpha_j p_j. \qquad (2)$$

The 13-value input s contains each expert's entropy divided by $\log 4$, top-two margin and maximum score, three pairwise total-variation distances, and global/crop embedding cosine distance. Training means and standard deviations standardize these values, with a 0.05 lower bound on standard deviation. The gate has 42 coefficients including biases. It adds no encoder pass and leaves the CNN and original heads unchanged.

To avoid teaching the gate from expert predictions on their own fitting examples, we divide only the 221 training groups into five stratified folds, seed 604. Each expert fits four folds and predicts the fifth. These 2,834 out-of-fold examples train the gate by class-balanced negative log likelihood plus $\lambda\|B\|_F^2/2$. Gate biases are unpenalized. Expert $C = 10$ comes from the earlier validation study, so this is not nested cross-validation. All expert scores use $T = 1$ here; validation-fitted temperatures never enter out-of-fold training. At evaluation, the original heads fitted on all training images supply scores.

We save the protocol before fitting and compare uniform averaging with a constant-weight gate trained on the same out-of-fold scores. Only the adaptive gate sees s. Validation macro F1 selects $\lambda \in \{0.01, 0.1, 1, 10\}$, preferring larger values on ties. The choice freezes before test evaluation. Since the original test results were already known, this remains an exploratory follow-up.

Validation selects $\lambda = 10$ and 78.8% macro F1, equal to constant weighting and below concatenation's 79.6%. On test the adaptive and constant gates make identical top-label decisions: 618/945 correct, one fewer than contrast. Learned constant weights are approximately 0.176 global, zero crop and 0.824 contrast. The adaptive gate's strong regularization yields nearly the same weighting. Its paired accuracy difference from contrast is -0.11 points, with a visual-group bootstrap interval of $[-0.79, +0.68]$. The experiment does not support input-dependent weighting on this split. The recorded validation rule therefore retains the existing demo model. Stored folds, gate coefficients and individual predictions permit this negative result to be checked.

5.4. Implementation and demonstration

Training uses scikit-learn; inference uses CPU ONNX Runtime and numeric NPZ/JSON heads. Shared functions implement training and demo preprocessing. The image page shows both actual inputs, four scores, review reasons and a HOG comparison. Uploads use temporary storage; inference needs no cloud service or API key after installation. JSON export identifies temperature-scaled softmax scores. Five shipped JPEG illustrations cover successes in all four classes and a confident error, with fresh predictions on their actual bytes. JPEG encoding can differ from the original benchmark input. The full local package adds the original view-disagreement image; a public clone can restore it optionally. All demo cases were selected after evaluation and do not define a new test set.

Warm CNN plus HOG inference takes 23 ms median and 52 ms at the 95th percentile over 50 fixed test images on the recorded host. This includes separate image decodes, but excludes initialization, HTTP transport and preview encoding. Preserved older CNN-only timings describe a different operation.

6. Limitations and conclusion

The project implements the course's feature-extractor/classifier structure and tests small two-view extensions. HOG gives an explicit gradient baseline; pretrained features work better on this fixed benchmark. Contrast changes decisions in useful and harmful directions, with only three net additional correct test predictions and uncertainty spanning no gain. The learned-weight follow-up also fails to improve on simpler fusion. The evidence supports a reproducible comparison and a transparent demo, without establishing superior recognition or a state-of-the-art method.

The duplicate-audit effect is an evaluation warning, not proof of independent groups. Unknown match identities, unverified upstream labels, validation reuse and exploratory development limit generalization claims. An independently labeled test collection with known match/player identities would answer a more useful question than searching this split for a higher score. The weak video study remains in Appendix A.3.

References

[1] amittalmale. CRICKET_SHOTS, version 1. Kaggle dataset, 2025. Accessed 17 October 2026.

[2] Aneesh. Cricket shots dataset, version 2. Kaggle dataset, 2022. Accessed 17 October 2026; uploader identifier aneesh10.

[3] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *CVPR*, pages 886–893, 2005.

[4] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *NeurIPS*, 2017.

[5] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML*, pages 1321–1330, 2017.

[6] Sk. Wasiqul Hoque, Md Ahsan Habib, Atanu Shome, and Sadman Sakib. KUCricShot: A dataset for developing detection model of cricket shots. In *ICCIT*, 2023. Dataset description checked against the authors' repository.

[7] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

[8] Sungwoo Kang. Modern deep learning approaches for cricket shot classification: A comprehensive baseline study. arXiv:2510.09187, preprint, 2025.

[9] ONNX Model Zoo contributors. MobileNetV2 pretrained onnx models. Accessed 17 October 2026.

[10] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *CVPR*, pages 4510–4520, 2018.

[11] A. Sen, K. Deb, P. K. Dhar, and T. Koshiba. CricShotClassify: An approach to classifying batting shots from cricket videos using a convolutional neural network and gated recurrent unit. *Sensors*, 21(8):2846, 2021.

[12] Limin Wang, Yuanjun Xiong, Zhe Wang, Yu Qiao, Dahua Lin, Xiaoou Tang, and Luc Van Gool. Temporal segment networks: Towards good practices for deep action recognition. In *ECCV*, 2016.

A. Supplementary evidence

A.1. Visual-copy audit

The initial audit uses decoded RGB hashes, 64-bit perceptual hash Hamming distance at most six, and 16 by 16 RGB thumbnail mean absolute difference at most 18. Exact copies collapse; conflicting-label components are quarantined. The stronger audit adds quarter-turn/mirror hashes (same bit cutoff, aligned thumbnail difference at most 24), then proposes the nearest 15 frozen-feature neighbors within cosine distance 0.20.

ORB binary descriptors and a RANSAC homography verify proposed geometric links. Requirements are a 0.72 match-ratio cutoff, at least 16 filtered matches, 12 inliers, 65% inlier fraction and 8% convex-hull coverage in both images. This adds 948 transform and 1,797 geometric edges. There are no final conflicting components. All linked copies and initial hash components stay in one split. These checks reduce leakage but can still miss related photographs or join visually similar ones. Full criteria and preserved edges are in the commented audit script and data directory.

A.2. Secondary diagnostics

Scores	NLL	ECE	Brier
Unscaled	0.859	0.043	0.470
Scaled	0.898	0.094	0.482

Table 7. Unchanged contrast scores on test; lower is better. Scaling worsens NLL and ECE.

The original temperature is approximately 0.802. Validation is reused for regularization, temperature and review selection. The review search chooses the lowest threshold from 0.50 to 0.95 in steps of 0.05 attaining 90% validation accuracy with at least 20 accepted images and 20% coverage, with 0.80 fallback. Its chosen 0.70 threshold stays unchanged. At that threshold confidence alone covers 62.2% with 77.6% accepted accuracy; adding agreement covers 58.9% with 77.7%. This modest difference is descriptive, not a guaranteed improvement.

Head	Clean	Dark	Blur	Mirror
CNN global	63.0	62.8	48.0	62.9
Contrast	65.5	67.1	49.7	65.2

Table 8. Preserved descriptive stress-test accuracy in percent; no retraining or parameter choice.

Dark multiplies source brightness by 0.6; blur uses Gaussian radius two at source resolution; mirror reflects horizontally while retaining the four broad labels. Both CNN views are recomputed. These conditions do not represent a new match or camera, and blur severity varies with source resolution. Original predictions, stress evidence and CNN-only timings remain under `results/images/`; preliminary hash-only evidence remains under `results/images_initial_hash/`.

A.3. Exploratory video pilot

The separate Kaggle CRICKET_SHOTS archive [1] contains 562 clips in eight labels and 25 normalized source-title groups. Its test set has 116 clips from five groups. Nine uniformly sampled frames feed the frozen encoder. Three ordered phase means, late-minus-early difference and temporal standard deviation form 6,400 features. This sparse temporal summary is motivated by segment-based representations [12], without reproducing TSN training.

The full-frame pilot obtains 18.1% accuracy. A fixed upper-center crop, introduced after pilot test feedback, reaches 30.2% accuracy and 21.3% macro F1. The training-majority baseline also reaches 30.2% accuracy (5.8% macro F1). Equal majority accuracy is not a successful general video recognizer. Defence has only three test clips; validation has one unorthodox clip. The reused holdout makes this exploratory. Original and revised predictions remain available; different data and labels prevent a direct image-versus-video comparison.

A.4. Code, data and repositories

The report appends all submitted implementation and configuration files. Public libraries, pretrained weights and the CVPR style are not original code. The repository includes dependency files, dataset manifests, numeric heads, saved predictions, tests and reproduction commands. Shared functions and source comments document preprocessing and classifier behaviour.

Project repository: `https://github.com/hrushikeshroop/cricket-shot-recognition-ee604`.

Data: Cricket Shots Dataset v2 and CRICKET_SHOTS v1. Pretrained weights: ONNX Model Zoo. Libraries: scikit-learn, ONNX Runtime, OpenCV, Flask, and NumPy. Complete dependency links and attribution accompany `docs/sources.md`. Our NumPy HOG follows the cited method and supplied lectures; no external cricket classifier code was imported.

Original code and configuration. Line numbers identify source lines. Wrapped lines continue without a number.

app.py

```
"""Local-only Flask demo. No uploads or predictions leave this computer."""

from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parent
sys.path.insert(0, str(ROOT / "src"))
import base64
import csv
import io
import json
import tempfile
import threading
import numpy as np
from PIL import Image, ImageOps
from flask import Flask, jsonify, render_template, request, send_file
from werkzeug.exceptions import RequestEntityTooLarge
from cricketshot.core import Recognizer, DISPLAY, crop_frame
from cricketshot.images import ImageRecognizer
from cricketshot.hog import HOGRecognizer


def create_app(recognizer=None):
    app = Flask(__name__)
    app.config["MAX_CONTENT_LENGTH"] = 40 * 1024 * 1024
    inference_lock = threading.Lock()
    model_holder = [recognizer]
    image_model_holder = [None]
    hog_model_holder = [None]

    def model():
        if model_holder[0] is None:
            model_holder[0] = Recognizer()
        return model_holder[0]

    def examples():
        path = ROOT / "demo/examples.json"
        entries = json.loads(path.read_text(encoding="utf-8")) if path.exists() else []
        return [e for e in entries if (ROOT / e["path"]).is_file() and (ROOT / e["thumbnail"]).is_file()]

    def infer(path):
        with inference_lock:
            prediction, frames = model().predict(path)
        previews = []
        for frame in frames:
            im = ImageOps.fit(
                crop_frame(frame, model().config.get("view", "batter_roi")),
                (224, 224),
                method=Image.Resampling.BILINEAR,
            )
            im.thumbnail((240, 180))
            buffer = io.BytesIO()
            im.save(buffer, format="JPEG", quality=85)
            previews.append(
                ""
                + base64.b64encode(buffer.getvalue()).decode("ascii")
            )
        prediction["previews"] = previews
        return prediction

    @app.get("/")
```

app.py

```
    def index():
        return render_template("images.html")

    @app.context_processor
    def project_identity():
        return {"submission": json.loads((ROOT / "configs/submission.json").read_text(encoding="utf-8"))}

    @app.get("/video")
    def video_index():
        return render_template("index.html", classes=DISPLAY)

    def image_examples():
        p = ROOT / "demo/image_examples.json"
        entries = json.loads(p.read_text(encoding="utf-8")) if p.exists() else []
        return [e for e in entries if (ROOT / e["path"]).is_file()]

    def infer_image(path):
        with inference_lock:
            if image_model_holder[0] is None:
                image_model_holder[0] = ImageRecognizer()
            result, views = image_model_holder[0].predict(path)
            if hog_model_holder[0] is None:
                hog_model_holder[0] = HOGRecognizer()
            result["hog_baseline"] = hog_model_holder[0].predict(path)
        result["previews"] = []
        for view in views:
            im = ImageOps.fit(view, (224, 224), method=Image.Resampling.BILINEAR)
            buffer = io.BytesIO()
            im.save(buffer, format="PNG")
            result["previews"].append(
                ""
                + base64.b64encode(buffer.getvalue()).decode("ascii")
            )
        return result

    @app.get("/image-api/examples")
    def image_example_list():
        return jsonify(
            [{k: v for k, v in e.items() if k != "path"} for e in image_examples()]
        )

    @app.get("/image-api/example/<example_id>/thumbnail")
    def image_thumbnail(example_id):
        entry = next((e for e in image_examples() if e["id"] == example_id), None)
        if not entry:
            return jsonify(error="Example not found"), 404
        return send_file(ROOT / entry["path"])

    @app.post("/image-api/example/<example_id>")
    def predict_image_example(example_id):
        entry = next((e for e in image_examples() if e["id"] == example_id), None)
        if not entry:
            return jsonify(error="Example not found"), 404
        result = infer_image(ROOT / entry["path"])
        result["example"] = {k: v for k, v in entry.items() if k != "path"}
        return jsonify(result)

    @app.post("/image-api/predict")
    def predict_image_upload():
        uploaded = request.files.get("image")
        if not uploaded or not uploaded.filename:
            return jsonify(error="Choose an image first."), 400
        suffix = Path(uploaded.filename).suffix.lower()
        if suffix not in {".jpg", ".jpeg", ".png", ".webp", ".bmp"}:
```

app.py

```
            return jsonify(error="Use JPEG, PNG, WebP or BMP."), 400
        with tempfile.TemporaryDirectory(prefix="cricketshot_image_") as folder:
            path = Path(folder) / ("image" + suffix)
            uploaded.save(path)
            return jsonify(infer_image(path))

    @app.get("/image-api/evidence")
    def image_evidence():
        return jsonify(
            json.loads(
                (ROOT / "results/images/metrics.json").read_text(encoding="utf-8")
            )
        )

    @app.get("/api/status")
    def status():
        ready = all(
            (ROOT / "models" / f).exists()
            for f in ("classifier.json", "classifier.npz", "mobilenetv2_features.onnx")
        )
        image_ready = all(
            (ROOT / f).exists()
            for f in (
                "models/image/classifier.json",
                "models/image/heads.npz",
                "models/hog/head.npz",
                "models/mobilenetv2_features.onnx",
            )
        )
        return jsonify(
            ready=image_ready,
            video_pilot_ready=ready,
            version="2.0.0",
            scope="One image, four dataset classes; video is an exploratory backup",
        )

    @app.get("/api/evidence")
    def evidence():
        return jsonify(
            json.loads((ROOT / "results/metrics.json").read_text(encoding="utf-8"))
        )

    @app.get("/api/examples")
    def example_list():
        return jsonify(
            [{k: v for k, v in e.items() if k != "path"} for e in examples()]
        )

    @app.get("/api/example/<example_id>/thumbnail")
    def thumbnail(example_id):
        entry = next((e for e in examples() if e["id"] == example_id), None)
        if not entry:
            return jsonify(error="Example not found"), 404
        return send_file(ROOT / entry["thumbnail"], mimetype="image/jpeg")

    @app.post("/api/example/<example_id>")
    def predict_example(example_id):
        entry = next((e for e in examples() if e["id"] == example_id), None)
        if not entry:
            return jsonify(error="Example not found"), 404
        result = infer(ROOT / entry["path"])
        result["example"] = {k: v for k, v in entry.items() if k != "path"}
        return jsonify(result)

```

app.py

```
    @app.post("/api/predict")
    def predict_upload():
        uploaded = request.files.get("video")
        if not uploaded or not uploaded.filename:
            return jsonify(error="Choose a trimmed video first."), 400
        suffix = Path(uploaded.filename).suffix.lower()
        if suffix not in {".avi", ".mp4", ".mov", ".mkv", ".webm"}:
            return jsonify(error="Use an AVI, MP4, MOV, MKV or WebM video."), 400
        # A private temporary file avoids trusting user-supplied filenames.
        with tempfile.TemporaryDirectory(prefix="cricketshot_") as folder:
            path = Path(folder) / ("upload" + suffix)
            uploaded.save(path)
            return jsonify(infer(path))

    @app.errorhandler(ValueError)
    def invalid_video(exc):
        return jsonify(error=str(exc)), 400

    @app.errorhandler(FileNotFoundError)
    def missing_model(exc):
        return (
            jsonify(
                error="Model files are missing. Follow the setup steps in README.md."
            ),
            503,
        )

    @app.errorhandler(RequestEntityTooLarge)
    def too_large(exc):
        return (
            jsonify(error="The upload limit is 40 MB. Trim or compress the clip."),
            413,
        )

    return app


if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser()
    parser.add_argument("--port", type=int, default=6040)
    args = parser.parse_args()
    create_app().run(host="127.0.0.1", port=args.port, debug=False, threaded=True)
```

configs/experiment.json

```
{
  "seed": 17,
  "frames": 9,
  "split": "5 source-grouped folds: test=0, val=1, train=2,3,4",
  "descriptors": ["middle", "mean", "phase", "phase_delta"],
  "C_candidates": [0.1, 1, 10, 100],
  "selection_metric": "validation macro F1, smallest C wins ties",
  "deployment_descriptor": "phase_delta",
  "view": "batter_roi",
  "temperature_bounds": [0.1, 10.0],
  "review_target_validation_accuracy": 0.9,
  "review_min_validation_coverage": 0.2,
  "bootstrap_source_resamples": 2000,
  "comment": "The original full-frame specification preceded pilot test scores. The ROI revision followed the failed pilot and keeps the same split, so final results are exploratory on a reused holdout. ROI bounds were fixed before ROI scoring. The proposed model remains phase_delta even if an ablation scores higher."
```

configs/experiment.json

```
}
```

configs/fusion_experiment.json

```
{
  "seed": 604,
  "experts": ["global", "focus", "contrast"],
  "expert_C": 10,
  "expert_temperature": 1.0,
  "training_folds": 5,
  "gate_l2_candidates": [0.01, 0.1, 1.0, 10.0],
  "selection": "Highest validation macro F1; larger L2 wins exact ties. Test is not used for selection.",
  "deployment_rule": "Retain contrast unless adaptive fusion exceeds every existing head and both fusion controls on validation macro F1.",
  "controls": ["uniform_mean", "constant_weights"],
  "objective": "Class-balanced training out-of-fold negative log likelihood plus 0.5*L2*sum(gate slopes squared). Gate intercepts are unpenalized.",
  "features": "For each expert: entropy/log(4), top-two margin, maximum score; three pairwise total-variation distances; cosine distance of normalized global/crop embeddings. Standardize with training out-of-fold means and std clipped below at 0.05.",
  "caution": "Exploratory follow-up after the existing test results were known. Expert C=10 was selected in the earlier validation study; this is not nested cross-validation. Mixtures of experts are established, not a new architecture."
}
```

configs/hog_experiment.json

```
{
  "status": "Exploratory extension after seeing Option B results; fixed before the new HOG experiment",
  "question": "How does a hand-crafted gradient descriptor compare with frozen CNN features on the identical images and split?",
  "feature": "224 square global-view luminance, unsigned 9-bin HOG, 16x16 cells, 2x2-cell blocks, stride one cell, L2 block and image normalization",
  "split_policy": "Reuse existing manifest, test identities and groups exactly; no split/seed search",
  "C_candidates": [0.1, 1, 10, 100],
  "selection": "Validation macro F1; smallest C wins ties",
  "classifier": "L2-regularized multinomial logistic regression, balanced class weights, lbfgs, max_iter=2000",
  "scaling": "Deterministic per-image normalization only. No learned scaling, PCA or feature selection.",
  "calibration": "None for HOG. Existing CNN temperatures and review threshold remain unchanged.",
  "uncertainty": "2000 paired visual-group bootstrap draws, seed 17, fixed model and split; no independence claim for 945 images",
  "fusion": "No HOG+CNN fusion experiment; keep one focused course comparison"
}
```

configs/image_experiment.json

```
{
  "seed": 604,
  "views": ["full image", "crop x=15..85%, y=5..95%"],
  "modes": ["global", "focus", "concat", "contrast"],
  "deployment_mode": "contrast",
  "duplicate_rule": "Initial RGB/pHash groups strengthened with quarter-turn/mirror pHash and ORB/RANSAC verified frozen-feature nearest-neighbor pairs. See strengthen_image_groups.py for exact criteria. Components stay in one split.",
  "split": "StratifiedGroupKFold(5, shuffle=True, random_state=604): test fold0, val fold1, train other folds",
  "C_candidates": [0.1, 1, 10, 100],
  "selection": "validation macro F1, smallest C wins ties",
  "review": "Lowest threshold 0.50..0.95 step0.05 with agreement of global, focus and fused heads, at least 20 accepted validation images, >=20% validation coverage, >=90% validation accuracy. Fallback threshold=0.80.",
  "status": "The view geometry and head comparison preceded all image scores. Visual QA after the initial hash-grouped score found transformed copies; grouping was strengthened and the comparison rerun. Final
```

configs/image_experiment.json

```
    revised-grouping scores are exploratory, with initial scores preserved. No source/match IDs are available."
}
```

configs/submission.json

```
{
  "group_name": "EE604 Project Group",
  "github_url": "https://github.com/hrushikeshroop/cricket-shot-recognition-ee604",
  "member_names": [
    "Hrushikesh Roop Avvari",
    "Voora Rakesh",
    "Gokavarapu Manikanta Sri Akshith",
    "Lokesh Kumar Moond",
    "Bollu Sharath chandra"
  ],
  "members": [
    {
      "name": "Hrushikesh Roop Avvari",
      "roll_number": "230249"
    },
    {
      "name": "Voora Rakesh",
      "roll_number": "231174"
    },
    {
      "name": "Gokavarapu Manikanta Sri Akshith",
      "roll_number": "230418"
    },
    {
      "name": "Lokesh Kumar Moond",
      "roll_number": "230606"
    },
    {
      "name": "Bollu Sharath chandra",
      "roll_number": "230303"
    }
  ]
}
```

pyproject.toml

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "cricketshot-ee604"
version = "1.0.0"
description = "Four-class cricket image recognition with HOG and frozen CNN features"
requires-python = ">=3.11"
dependencies = ["numpy>=1.26", "Pillow>=10", "onnxruntime>=1.17", "opencv-python-headless>=4.9", "Flask>=3.0"]

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
```

requirements-dev.txt

requirements-dev.txt

```
-r requirements.txt
scikit-learn>=1.4
scipy>=1.12
threadpoolctl>=3.2
onnx>=1.15
ImageHash>=4.3
matplotlib>=3.8
requests>=2.31
pytest>=8.0
reportlab>=4.0
PyMuPDF>=1.24
```

requirements-lock.txt

```
# Exact versions used on Python 3.14.5 / Windows x64.
blinker==1.9.0
certifi==2026.7.22
charset-normalizer==3.5.2
click==8.5.0
colorama==0.4.6
contourpy==1.4.0
cycler==0.12.1
Flask==3.1.3
flatbuffers==25.12.19
fonttools==4.66.1
idna==3.20
ImageHash==4.3.2
iniconfig==2.3.1
itsdangerous==2.2.0
Jinja2==3.1.6
joblib==1.6.0
kiwisolver==1.5.1
MarkupSafe==3.0.4
matplotlib==3.11.2
ml_dtypes==0.6.0
numpy==2.5.3
onnx==1.23.2
onnxruntime==1.31.0
opencv-python-headless==5.0.0.93
packaging==26.3
Pillow==12.3.0
pluggy==1.7.0
protobuf==7.36.2
Pygments==2.21.0
PyMuPDF==1.28.2
pyparsing==3.3.3
pytest==9.1.1
python-dateutil==2.9.0.post0
PyWavelets==1.10.0
reportlab==5.0.1
requests==2.34.2
scikit-learn==1.9.1
scipy==1.18.1
six==1.17.0
threadpoolctl==3.7.0
urllib3==2.8.0
Werkzeug==3.1.9
```

requirements.txt

requirements.txt

```
# Demo and inference, Python 3.11 or newer. See requirements-lock.txt for tested versions.
numpy>=1.26
Pillow>=10
onnxruntime>=1.17
opencv-python-headless>=4.9
Flask>=3.0
```

run_demo.bat

```
@echo off
cd /d "%~dp0"
if not exist ".venv\Scripts\python.exe" (
  echo Run setup.bat first.
  pause
  exit /b 1
)
echo CricketShot will be available at http://127.0.0.1:6040
".venv\Scripts\python.exe" app.py
pause
```

scripts/analyze_revision.py

```
"""Recompute comparisons and paired uncertainty from saved predictions only."""
from pathlib import Path
import sys
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import hashlib
import json
import numpy as np
from train_evaluate import metrics


def confusion_statistics(matrix):
    """Accept (..., 4, 4) counts and compute image-weighted accuracy/macro F1."""
    diagonal = np.diagonal(matrix, axis1=-2, axis2=-1)
    accuracy = diagonal.sum(-1) / matrix.sum(axis=(-2, -1))
    denominator = matrix.sum(-1) + matrix.sum(-2)
    f1 = np.divide(2 * diagonal, denominator, out=np.zeros_like(diagonal, dtype=float),
                   where=denominator > 0).mean(-1)
    return np.stack([accuracy, f1], axis=-1)


def describe_view_changes(ids, y, groups, predictions, classes):
    """Post-hoc diagnostics of fixed predictions; no model/threshold is selected."""
    labels = {mode: predictions[mode].argmax(1) for mode in ("global", "focus", "concat", "contrast")}
    agree = labels["global"] == labels["focus"]
    concat_ok = labels["concat"] == y
    contrast_ok = labels["contrast"] == y
    gained = contrast_ok & ~concat_ok
    lost = concat_ok & ~contrast_ok
    strata = {}
    for name, mask in [("all", np.ones(len(y), dtype=bool)), ("views_agree", agree), ("views_disagree",
~agree)]:
        strata[name] = {"n": int(mask.sum()), "concat_correct": int((concat_ok & mask).sum()),
                        "contrast_correct": int((contrast_ok & mask).sum()),
                        "gained": int((gained & mask).sum()), "lost": int((lost & mask).sum())}
    cases = []
```

scripts/analyze_revision.py

```
    for role, mask in [("contrast_correct_concat_wrong", gained), ("concat_correct_contrast_wrong", lost)]:
        # Stable ID order, not a confidence search. These cases illustrate both directions.
        chosen = sorted(np.flatnonzero(mask), key=lambda i: str(ids[i]))[:2]
        for i in chosen:
            cases.append({"id": str(ids[i]), "group": str(groups[i]), "role": role,
                          "dataset_label": classes[int(y[i])], "single_view_heads_agree": bool(agree[i]),
                          "predicted_labels": {mode: classes[int(label[i])] for mode, label in labels.items()}})
    return {"scope": "Post-hoc descriptive comparison of unchanged held-out predictions; not a causal explanation or a model-selection step.",
            "strata": strata, "illustrative_cases": cases,
            "case_selection": "First two IDs in each gain/loss category; selected after evaluation, not representative frequency samples.",
            "interpretation": "The absolute-difference representation changes decisions in both directions. A small net count and conditional subsets do not establish reliable superiority or identify the cause of an individual error."}


def main():
    out = ROOT / "results/revision"
    out.mkdir(parents=True, exist_ok=True)
    old = json.loads((ROOT / "results/images/metrics.json").read_text())
    rows = list(csv.DictReader((ROOT / "data/images/processed/manifest.csv").open(encoding="utf-8", newline="")))
    test_rows = [r for r in rows if r["split"] == "test"]
    with np.load(ROOT / "results/images/test_predictions.npz", allow_pickle=False) as arrays:
        y, ids, groups = [arrays[k].copy() for k in ("y", "ids", "groups")]
        predictions = {mode: arrays["p_" + mode].copy() for mode in ("global", "focus", "concat", "contrast")}
        accepted = arrays["accepted"].copy()
    with np.load(ROOT / "results/hog/test_predictions.npz", allow_pickle=False) as arrays:
        assert np.array_equal(ids, arrays["ids"])
        assert np.array_equal(y, arrays["y"])
        predictions["hog"] = arrays["p"].copy()
    assert ids.tolist() == [r["id"] for r in test_rows]
    assert len(np.unique(groups)) == 70 and len(y) == 945
    classes = old["classes"]
    unique = np.unique(groups)
    modes = list(predictions)
    group_matrices = np.zeros((len(modes), len(unique), 4, 4), dtype=int)
    for mi, mode in enumerate(modes):
        pred = predictions[mode].argmax(1)
        for gi, group in enumerate(unique):
            index = np.flatnonzero(groups == group)
            np.add.at(group_matrices[mi, gi], (y[index], pred[index]), 1)
    rng = np.random.default_rng(17)
    # Resample whole visual groups. Every image of a drawn group is retained.
    # Use the identical draws for all methods to form paired differences.
    draws = np.stack([np.bincount(rng.choice(len(unique), size=len(unique), replace=True),
                                  minlength=len(unique)) for _ in range(2000)])
    boot_matrices = np.einsum("bg,mgij->mbij", draws, group_matrices)
    boot_values = confusion_statistics(boot_matrices)
    comparisons = {}
    for mi, mode in enumerate(modes):
        measured = metrics(y, predictions[mode], classes)
        if mode != "hog":
            assert abs(measured["accuracy"] - old["models"][mode]["test"]["accuracy"]) < 1e-12
            assert abs(measured["macro_f1"] - old["models"][mode]["test"]["macro_f1"]) < 1e-12
        interval = np.percentile(boot_values[mi], [2.5, 97.5], axis=0)
        comparisons[mode] = {"test": measured, "correct": int((predictions[mode].argmax(1) == y).sum()),
                             "accuracy_95": interval[:, 0].tolist(),
                             "macro_f1_95": interval[:, 1].tolist()}
    ci, cj = modes.index("contrast"), modes.index("concat")
    difference = boot_values[ci] - boot_values[cj]
    paired_interval = np.percentile(difference, [2.5, 97.5], axis=0)
    paired = {"comparison": "contrast minus concatenation", "correct_difference":
```

scripts/analyze_revision.py

```
    comparisons["contrast"]["correct"] - comparisons["concat"]["correct"],
 96               "accuracy_difference": comparisons["contrast"]["test"]["accuracy"] -
    comparisons["concat"]["test"]["accuracy"],
 97               "accuracy_difference_95": paired_interval[:, 0].tolist(),
 98               "macro_f1_difference_95": paired_interval[:, 1].tolist()}
 99     info = {"status": "Exploratory; prior results already known; no new test-based model choice",
100             "models": comparisons, "paired_contrast_concat": paired,
101             "uncertainty": {"unit": "visual duplicate group", "groups": len(unique), "draws": 2000, "seed": 17,
102                             "method": "percentile cluster bootstrap; image-weighted statistics within
    whole-group resamples",
103                             "limitations": "Conditions on fixed training, split and imperfect visual groups;
    omits training variability and unknown match correlation."},
104             "review": {"accepted": int(accepted.sum()),
105                        "correct": int(((predictions["contrast"].argmax(1) == y) & accepted).sum()),
106                        "coverage": float(accepted.mean()),
107                        "accepted_accuracy": float((predictions["contrast"].argmax(1)[accepted] ==
    y[accepted]).mean()),
108                        "threshold_unchanged": old["review"]["threshold"]},
109             "calibration": {"uncalibrated": old["models"]["contrast"]["test_uncalibrated"],
110                             "scaled": old["models"]["contrast"]["test"],
111                             "conclusion": "Test NLL and ECE worsened. Retain original temperature; no
    improvement claim."},
112             "manifest_sha256": hashlib.sha256((ROOT /
    "data/images/processed/manifest.csv").read_bytes()).hexdigest()}
113     (out / "comparison.json").write_text(json.dumps(info, indent=2))
114     diagnosis = describe_view_changes(ids, y, groups, predictions, classes)
115     diagnosis["prediction_archive_sha256"] = hashlib.sha256((ROOT /
    "results/images/test_predictions.npz").read_bytes()).hexdigest()
116     (out / "view_diagnostics.json").write_text(json.dumps(diagnosis, indent=2), encoding="utf-8")
117     with (out / "comparison.csv").open("w", newline="", encoding="utf-8") as stream:
118         writer = csv.writer(stream)
119         writer.writerow(["method", "correct", "n", "accuracy", "macro_f1", "accuracy_ci_low",
    "accuracy_ci_high", "f1_ci_low", "f1_ci_high"])
120         for mode, result in comparisons.items():
121             writer.writerow([mode, result["correct"], len(y), result["test"]["accuracy"],
    result["test"]["macro_f1"],
122                              *result["accuracy_95"], *result["macro_f1_95"]])
123     print(json.dumps({"paired": paired, "review": info["review"], "hog": comparisons["hog"]}, indent=2))
124
125
126 if __name__ == "__main__":
127     main()
```

scripts/benchmark.py

```
  1 """Time end-to-end CPU inference on deterministic held-out clips, without caches."""
  2 from pathlib import Path
  3 import sys
  4 ROOT=Path(__file__).resolve().parents[1]
  5 sys.path.insert(0,str(ROOT/'src'))
  6 import csv
  7 import json
  8 import platform
  9 import numpy as np
 10 from cricketshot.core import Recognizer
 11
 12 def main():
 13     rows=list(csv.DictReader((ROOT/'data/processed/manifest.csv').open(encoding='utf-8',newline='')))
 14     rows=[r for r in rows if r['split']=='test']
 15     model=Recognizer()
 16     model.predict(ROOT/rows[0]['path'])
 17     sample=[rows[i] for i in np.linspace(0,len(rows)-1,min(25,len(rows)),dtype=int)]
 18     times=[]
```

scripts/benchmark.py

```
    for row in sample:
        result,_=model.predict(ROOT/row['path']);times.append(result['seconds'])
    result={'clips':len(times),'warmup_clips':1,'scope':'Decode, sample, nine CNN passes and classifier. Model initialization excluded.',
            'median_seconds':float(np.median(times)),'p95_seconds':float(np.percentile(times,95)),
            'mean_seconds':float(np.mean(times)),'times_seconds':times,'clip_ids':[r['id'] for r in sample],
            'platform':platform.platform(),'processor':platform.processor(),'onnx_intra_op_threads':2}
    (ROOT/'results/latency.json').write_text(json.dumps(result,indent=2),encoding='utf-8')
    print(json.dumps(result,indent=2))

if __name__=='__main__':main()
```

scripts/extract_roi_features.py

```
"""Second, explicitly exploratory, experiment after observing the pilot failure.

The split is unchanged. ROI geometry was fixed before computing ROI validation
and test results. This is development on a reused holdout, not pristine testing.
"""
from pathlib import Path
import sys
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT/'src'))
import csv
import json
import shutil
import numpy as np
from cricketshot.core import Encoder, decode_video

def main():
    d=ROOT/'data/processed'
    # Preserve the full-frame experiment and never overwrite its evidence.
    original=d/'features_fullframe.npz'
    if not original.exists():shutil.copy2(d/'features.npz',original)
    cache=d/'roi_feature_cache';cache.mkdir(exist_ok=True)
    rows=list(csv.DictReader((d/'manifest.csv').open(encoding='utf-8',newline='')))
    encoder=Encoder(view='batter_roi');all_features=[]
    for i,row in enumerate(rows):
        cp=cache/(row['sha256']+'.npy')
        if cp.exists():feat=np.load(cp,allow_pickle=False)
        else:
            frames,_=decode_video(ROOT/row['path']);feat=encoder.encode(frames)
            np.save(cp,feat,allow_pickle=False)
        all_features.append(feat)
        if (i+1)%50==0:print(f'ROI features {i+1}/{len(rows)}',flush=True)
    with np.load(original,allow_pickle=False) as a:
        data={k:a[k] for k in a.files}
    data['features']=np.stack(all_features)
    np.savez_compressed(d/'features.npz',**data)
    record={'view':'batter_roi','normalized_bounds':[.30,.02,.70,.65],
            'motivation':'Full-frame pilot showed poor recognition. Small batters and broadcast context motivated a fixed upper-center crop.',
            'evaluation_status':'Exploratory reused holdout. Original test results were observed before this revision.',
            'split':'Unchanged source-grouped split, seed 17','detector':False}
    (ROOT/'results/roi_protocol.json').write_text(json.dumps(record,indent=2),encoding='utf-8')
    print('ROI features ready.',flush=True)

if __name__=='__main__':main()
```

scripts/fetch_assets.py

scripts/fetch_assets.py

```
"""Download public upstream assets, keeping URLs and checksums for provenance."""
from pathlib import Path
import hashlib
import json
import requests
import zipfile

ROOT = Path(__file__).resolve().parents[1]

def fetch(url, target, expected_sha256=None):
    target = ROOT / target
    target.parent.mkdir(parents=True, exist_ok=True)
    if not target.exists():
        with requests.get(url, stream=True, timeout=(30, 180)) as r:
            r.raise_for_status()
            with target.with_suffix(target.suffix + '.part').open('wb') as f:
                for chunk in r.iter_content(1024 * 1024):
                    f.write(chunk)
        target.with_suffix(target.suffix + '.part').replace(target)
    digest = hashlib.sha256(target.read_bytes()).hexdigest()
    if expected_sha256 and digest != expected_sha256:
        raise ValueError(f'Checksum mismatch for {target.name}. Refusing changed or incomplete asset.')
    print(target.relative_to(ROOT), target.stat().st_size, digest, flush=True)
    return target, {'url': url, 'path': str(target.relative_to(ROOT)), 'sha256': digest,
                    'bytes': target.stat().st_size, 'accessed': '2026-10-17'}

def main():
    provenance = []
    ref = 'amittalmale/cricket-shots'
    meta = requests.get('https://www.kaggle.com/api/v1/datasets/view/' + ref, timeout=60)
    meta.raise_for_status()
    (ROOT / 'data').mkdir(exist_ok=True)
    (ROOT / 'data/source_metadata.json').write_text(json.dumps(meta.json(), indent=2), encoding='utf-8')
    archive, item = fetch('https://www.kaggle.com/api/v1/datasets/download/' + ref + '?datasetVersionNumber=1',
'data/downloads/cricket-shots-v1.zip', '1b4ceb40a8c976c2a06b532aa0280b82fdf41cfc1d83cd228e9159cfa5b5053d')
    provenance.append(item)
    out = ROOT / 'data/raw'
    out.mkdir(exist_ok=True)
    with zipfile.ZipFile(archive) as z:
        for member in z.infolist():
            dest = (out / member.filename).resolve()
            if not dest.is_relative_to(out.resolve()):
                raise ValueError('Unsafe archive path')
            if not dest.exists():
                z.extract(member, out)
        print('Archive entries', len(z.infolist()), flush=True)
        print('\n'.join(z.namelist()[:10]), flush=True)
    _, item = fetch('https://media.githubusercontent.com/media/onnx/models/main/validated/vision/classification/
mobilenet/model/mobilenetv2-12.onnx', 'models/mobilenetv2-12.onnx',
'c0c3f76d93fa3fd6580652a45618618a220fced18babf65774ed169de0432ad5')
    provenance.append(item)
    (ROOT / 'data/provenance.json').write_text(json.dumps(provenance, indent=2), encoding='utf-8')

if __name__ == '__main__':
    main()
```

scripts/fetch_images.py

scripts/fetch_images.py

```
"""Fetch the public four-class still-image dataset and preserve its metadata."""
from pathlib import Path
import json
import zipfile
import requests
from fetch_assets import fetch
ROOT=Path(__file__).resolve().parents[1]

def main():
    ref='aneesh10/cricket-shot-dataset'
    meta=requests.get('https://www.kaggle.com/api/v1/datasets/view/'+ref,timeout=60);meta.raise_for_status()
    out=ROOT/'data/images';out.mkdir(parents=True,exist_ok=True)
    (out/'source_metadata.json').write_text(json.dumps(meta.json(),indent=2),encoding='utf-8')
    version=2
    archive,item=fetch('https://www.kaggle.com/api/v1/datasets/download/'+ref+f'?datasetVersionNumber={version}'
,f'data/downloads/cricket-images-v{version}.zip',
'9d497543aca86587c65e167b19f4704e1e639af61da0dbe30a3a7d1485d53e62')
    (out/'provenance.json').write_text(json.dumps(item,indent=2),encoding='utf-8')
    raw=out/'raw';raw.mkdir(exist_ok=True)
    with zipfile.ZipFile(archive) as z:
        for member in z.infolist():
            if not (raw/member.filename).resolve().is_relative_to(raw.resolve()):raise ValueError('Unsafe
archive path')
            if not (raw/member.filename).exists():z.extract(member,raw)
        print('Images archive entries',len(z.infolist()),z.namelist()[:5],flush=True)

if __name__=='__main__':main()
```

scripts/prepare_data.py

```
"""Audit real videos, establish source-disjoint splits, cache frozen CNN features."""
from pathlib import Path
import sys
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / 'src'))
import csv
import hashlib
import json
import re
from collections import Counter
import cv2
import imagehash
import numpy as np
import onnx
from PIL import Image
from sklearn.model_selection import StratifiedGroupKFold
from cricketshot.core import Encoder, decode_video, color_descriptor

def source_name(stem):
    name = re.sub(r'_\d+_\d+$', '', stem)
    name = re.sub(r'[^a-z0-9]+', ' ', name.lower()).strip()
    name = re.sub(r'\b(icc|wt20|match|highlights|cricket|semi|final)\b', '', name)
    name = re.sub(r'\bv\b', 'vs', name)
    name = re.sub(r'\s+', ' ', name).strip()
    # Two Ireland/Oman highlight titles differ only in v/vs and the word match.
    # Sorting team names also conservatively merges a reversed title pairing.
    return ' vs '.join(sorted(s.strip() for s in name.split(' vs ')))

def build_encoder():
    source = ROOT / 'models/mobilenetv2-12.onnx'
```

scripts/prepare_data.py

```
    out = ROOT / 'models/mobilenetv2_features.onnx'
    if out.exists():
        return
    model = onnx.load(source)
    # Expose the global pooled 1280-dimensional tensor before ImageNet's head.
    pools = [n for n in model.graph.node if n.op_type == 'GlobalAveragePool']
    if len(pools) != 1:
        raise ValueError('Unexpected upstream encoder graph')
    tensor = pools[0].output[0]
    onnx.utils.extract_model(str(source), str(out), [model.graph.input[0].name], [tensor])
    onnx.checker.check_model(onnx.load(out))

def main():
    out = ROOT / 'data/processed'
    out.mkdir(parents=True, exist_ok=True)
    build_encoder()
    encoder = Encoder(view='full')
    paths = sorted((ROOT / 'data/raw').rglob('*.avi'))
    if not paths:
        raise RuntimeError('No videos. Run python scripts/fetch_assets.py first.')
    rows, features, colors, hashes, errors = [], [], [], [], []
    cache = out / 'feature_cache'
    cache.mkdir(exist_ok=True)
    for n, path in enumerate(paths):
        sha = hashlib.sha256(path.read_bytes()).hexdigest()
        cp = cache / (sha + '.npz')
        try:
            if cp.exists():
                with np.load(cp, allow_pickle=False) as a:
                    feat, col, hs = a['features'], a['color'], a['phash']
                    meta = json.loads(str(a['metadata']))
            else:
                frames, meta = decode_video(path)
                feat, col = encoder.encode(frames), color_descriptor(frames)
                hs = np.array([str(imagehash.phash(Image.fromarray(frames[i]))) for i in (0, 4, 8)])
                np.savez_compressed(cp, features=feat, color=col, phash=hs, metadata=json.dumps(meta))
            rows.append({'id': f'clip_{n:04d}', 'path': path.relative_to(ROOT).as_posix(),
                         'label': path.parent.name, 'source': source_name(path.stem),
                         'original_name': path.name, 'sha256': sha,
                         'frames': meta['decoded_frames'], 'duration_seconds': meta['duration_seconds']})
            features.append(feat)
            colors.append(col)
            hashes.append([int(h, 16) for h in hs])
        except Exception as exc:
            errors.append({'path': path.relative_to(ROOT).as_posix(), 'error': str(exc)})
        if (n+1) % 25 == 0:
            print(f'Encoded {n+1}/{len(paths)} clips, errors={len(errors)}', flush=True)
    # Union source groups when exact or strongly similar clips cross source names.
    parent = list(range(len(rows)))
    def find(i):
        while parent[i] != i:
            parent[i] = parent[parent[i]]
            i = parent[i]
        return i
    def union(i, j):
        parent[find(j)] = find(i)
    sources, shas = {}, {}
    for i, row in enumerate(rows):
        for key, mapping in ((row['source'], sources), (row['sha256'], shas)):
            if key in mapping:
                union(i, mapping[key])
            else:
                mapping[key] = i
    near = []
```

scripts/prepare_data.py

```
    for i in range(len(rows)):
        for j in range(i):
            if rows[i]['source'] == rows[j]['source']:
                continue
            distances = [(a ^ b).bit_count() for a, b in zip(hashes[i], hashes[j])]
            if max(distances) <= 6 and np.mean(distances) <= 4:
                union(i, j)
                near.append({'a': rows[j]['id'], 'b': rows[i]['id'], 'distances': distances,
                             'label_conflict': rows[j]['label'] != rows[i]['label']})
    labels = np.array([r['label'] for r in rows])
    groups = np.array([find(i) for i in range(len(rows))])
    folds = list(StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=17).split(np.zeros(len(rows)),
labels, groups))
    split = np.full(len(rows), 'train', dtype='<U5')
    split[folds[0][1]] = 'test'
    split[folds[1][1]] = 'val'
    for i, row in enumerate(rows):
        row['group'] = f'source_{groups[i]:04d}'
        row['split'] = str(split[i])
    for label in sorted(set(labels)):
        for s in ('train', 'val', 'test'):
            if not any((labels == label) & (split == s)):
                raise ValueError(f'Class {label} absent from {s}')
    with (out / 'manifest.csv').open('w', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]))
        w.writeheader()
        w.writerows(rows)
    np.savez_compressed(out / 'features.npz', features=np.stack(features), color=np.stack(colors),
                        labels=labels, split=split, groups=groups, ids=np.array([r['id'] for r in rows]))
    audit = {'seed': 17, 'split_policy': 'StratifiedGroupKFold(5), test fold 0, validation fold 1, other folds
train',
             'upstream_files': len(paths), 'decoded_files': len(rows), 'errors': errors,
             'class_counts': dict(Counter(labels)), 'sources': len(sources), 'union_groups': len(set(groups)),
             'exact_duplicate_files': len(rows) - len(shas), 'cross_source_near_duplicates': near,
             'near_duplicate_rule': '64-bit pHash at first/middle/last sampled frame: max Hamming <=6 and mean
<=4',
             'split_counts': {s: dict(Counter(labels[split == s])) for s in ('train', 'val', 'test')},
             'split_groups': {s: len(set(groups[split == s])) for s in ('train', 'val', 'test')},
             'feature_shape': list(np.stack(features).shape), 'frame_sampling': '9 uniform samples, endpoints
included'}
    (out / 'audit.json').write_text(json.dumps(audit, indent=2), encoding='utf-8')
    print(json.dumps(audit, indent=2), flush=True)

if __name__ == '__main__':
    main()
```

scripts/prepare_demo_examples.py

```
"""Recreate the selected local examples without changing experiment evidence.

Five examples reuse the illustrations already shipped with the slides. Fetch
the pinned archives with --download only for the optional original disagreement
image and backup clips. Download requires requests. No training is performed.
"""
from pathlib import Path
import argparse
import csv
import json
import shutil
import subprocess
import sys

ROOT = Path(__file__).resolve().parents[1]
```

scripts/prepare_demo_examples.py

```
sys.path.insert(0, str(ROOT / "src"))
from cricketshot.images import load_image


def restore_images():
    with (ROOT / "data/images/processed/manifest.csv").open(encoding="utf-8", newline="") as stream:
        rows = {row["id"]: row for row in csv.DictReader(stream)}
    examples = json.loads((ROOT / "demo/image_examples.json").read_text())
    for entry in examples:
        if entry.get("bundled"):
            if not (ROOT / entry["path"]).is_file():
                raise FileNotFoundError("Bundled slide illustration missing: "+entry["path"])
            continue
        row = rows[entry["id"]]
        if row["label"] != entry["label"] or row["split"] != entry["split"]:
            raise ValueError("Example metadata does not match the recorded manifest")
        source = ROOT / row["path"]
        if not source.is_file():
            raise FileNotFoundError("Raw image missing. Run this script with --download first.")
        target = ROOT / entry["path"]
        target.parent.mkdir(parents=True, exist_ok=True)
        load_image(source).save(target)
    return len(examples)


def restore_videos():
    from PIL import Image
    from cricketshot.core import decode_video
    with (ROOT / "data/processed/manifest.csv").open(encoding="utf-8", newline="") as stream:
        rows = {row["id"]: row for row in csv.DictReader(stream)}
    examples = json.loads((ROOT / "demo/examples.json").read_text())
    for entry in examples:
        source = ROOT / rows[entry["id"]]["path"]
        if not source.is_file():
            raise FileNotFoundError("Raw clip missing. Use --download --include-video.")
        target = ROOT / entry["path"]
        target.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(source, target)
        thumbnail = ROOT / entry["thumbnail"]
        thumbnail.parent.mkdir(parents=True, exist_ok=True)
        if not thumbnail.exists():
            frames, _ = decode_video(target)
            im = Image.fromarray(frames[len(frames)//2])
            im.thumbnail((320, 180))
            im.save(thumbnail, quality=85)
    return len(examples)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--download", action="store_true")
    parser.add_argument("--include-video", action="store_true")
    args = parser.parse_args()
    if args.download:
        subprocess.run([sys.executable, str(ROOT / "scripts/fetch_images.py")], cwd=ROOT, check=True)
        if args.include_video:
            subprocess.run([sys.executable, str(ROOT / "scripts/fetch_assets.py")], cwd=ROOT, check=True)
    print("Restored image examples:", restore_images())
    if args.include_video:
        print("Restored video examples:", restore_videos())


if __name__ == "__main__":
    main()
```

scripts/prepare_images.py

scripts/prepare_images.py

```
"""Audit, deduplicate, split and cache two-view features for real cricket images."""

from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import hashlib
import json
from collections import Counter
import imagehash
import numpy as np
from sklearn.model_selection import StratifiedGroupKFold
from cricketshot.core import Encoder
from cricketshot.images import load_image, image_views


def main():
    out = ROOT / "data/images/processed"
    out.mkdir(parents=True, exist_ok=True)
    cache = out / "cache"
    cache.mkdir(exist_ok=True)
    encoder = Encoder(view="full")
    paths = sorted(
        p
        for p in (ROOT / "data/images/raw").rglob("*")
        if p.suffix.lower() in (".jpg", ".jpeg", ".png", ".webp", ".bmp")
    )
    rows = []
    features = []
    phashes = []
    thumbs = []
    errors = []
    aliases = {
        "drives": "drive",
        "drive": "drive",
        "legglance-flick": "legglance-flick",
        "pullshot": "pullshot",
        "sweep": "sweep",
    }
    seen = {}
    for n, path in enumerate(paths):
        label = aliases.get(path.parent.name.lower())
        if not label:
            continue
        try:
            image = load_image(path)
            sha = hashlib.sha256(
                image.size[0].to_bytes(4, "big")
                + image.size[1].to_bytes(4, "big")
                + image.tobytes()
            ).hexdigest()
            views = image_views(image)
            cp = cache / (sha + ".npy")
            if cp.exists():
                feat = np.load(cp, allow_pickle=False)
            else:
                feat = encoder.encode([np.asarray(v) for v in views])
                np.save(cp, feat, allow_pickle=False)
            rows.append(
```

scripts/prepare_images.py

```
                {
                    "id": f"image_{n:05d}",
                    "path": path.relative_to(ROOT).as_posix(),
                    "label": label,
                    "rgb_sha256": sha,
                    "width": image.width,
                    "height": image.height,
                }
            )
            features.append(feat)
            phashes.append(int(str(imagehash.phash(image)), 16))
            thumbs.append(np.asarray(image.resize((16, 16)), dtype=np.int16))
        except Exception as exc:
            errors.append({"path": str(path.relative_to(ROOT)), "error": str(exc)})
        if (n + 1) % 200 == 0:
            print(f"Image features {n+1}/{len(paths)}, valid {len(rows)}", flush=True)
    if not rows:
        raise ValueError("No labeled images found. Run fetch_images.py first.")
    parent = list(range(len(rows)))

    def find(i):
        while parent[i] != i:
            parent[i] = parent[parent[i]]
            i = parent[i]
        return i

    def union(i, j):
        parent[find(j)] = find(i)

    exact = 0
    near = []
    for i, row in enumerate(rows):
        if row["rgb_sha256"] in seen:
            union(i, seen[row["rgb_sha256"]])
            exact += 1
        else:
            seen[row["rgb_sha256"]] = i
    for i in range(len(rows)):
        for j in range(i):
            if (phashes[i] ^ phashes[j]).bit_count() <= 6:
                mad = float(np.abs(thumbs[i] - thumbs[j]).mean())
                if mad <= 18:
                    union(i, j)
                    if rows[i]["rgb_sha256"] != rows[j]["rgb_sha256"]:
                        near.append([i, j, mad])
    components = {}
    for i in range(len(rows)):
        components.setdefault(find(i), []).append(i)
    conflicts = [
        ii for ii in components.values() if len({rows[i]["label"] for i in ii}) > 1
    ]
    excluded = {i for ii in conflicts for i in ii}
    # Collapse decoded exact duplicates to one representative within each label.
    representatives = []
    hash_seen = set()
    for i, row in enumerate(rows):
        if i not in excluded and row["rgb_sha256"] not in hash_seen:
            hash_seen.add(row["rgb_sha256"])
            representatives.append(i)
    kept = [rows[i] for i in representatives]
    feats = np.stack([features[i] for i in representatives])
    labels = np.array([r["label"] for r in kept])
    groups = np.array([find(i) for i in representatives])
    folds = list(
```

scripts/prepare_images.py

```
        StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=604).split(
            np.zeros(len(kept)), labels, groups
        )
    )
    split = np.full(len(kept), "train", dtype="<U5")
    split[folds[0][1]] = "test"
    split[folds[1][1]] = "val"
    for i, row in enumerate(kept):
        row["group"] = f"visual_{groups[i]:05d}"
        row["split"] = str(split[i])
    with (out / "manifest.csv").open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(kept[0]))
        w.writeheader()
        w.writerows(kept)
    np.savez_compressed(
        out / "features.npz",
        features=feats,
        labels=labels,
        split=split,
        groups=groups,
        ids=np.array([r["id"] for r in kept]),
    )
    audit = {
        "files_found": len(paths),
        "decoded_labeled": len(rows),
        "errors": errors,
        "exact_duplicate_excess": exact,
        "near_duplicate_pairs": len(near),
        "conflicting_components": len(conflicts),
        "conflicting_images_excluded": len(excluded),
        "exact_representatives_kept": len(kept),
        "visual_groups": len(set(groups)),
        "class_counts": dict(Counter(labels)),
        "split_counts": {
            s: dict(Counter(labels[split == s])) for s in ("train", "val", "test")
        },
        "split_groups": {
            s: len(set(groups[split == s])) for s in ("train", "val", "test")
        },
        "split_seed": 604,
        "source_ids_available": False,
        "rule": "Exact decoded RGB hash or pHash <=6 and thumbnail RGB MAD <=18",
        "warning": "Visual groups do not establish match/player independence. Related frames may remain across
splits.",
    }
    (out / "audit.json").write_text(json.dumps(audit, indent=2), encoding="utf-8")
    print(json.dumps(audit, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

scripts/strengthen_image_groups.py

```
"""Audit transformed copies before the final image comparison.

Initial hash-only scores are preserved. The final grouping adds dihedral pHash
and geometric ORB verification of frozen-feature nearest-neighbor candidates.
The protocol revision follows visual QA, so final scores are exploratory.
"""

from pathlib import Path
import sys
```

scripts/strengthen_image_groups.py

```
10
11  ROOT = Path(__file__).resolve().parents[1]
12  sys.path.insert(0, str(ROOT / "src"))
13  import csv
14  import json
15  import shutil
16  import cv2
17  import imagehash
18  import numpy as np
19  from PIL import Image, ImageOps
20  from sklearn.model_selection import StratifiedGroupKFold
21  from sklearn.neighbors import NearestNeighbors
22  from threadpoolctl import threadpool_limits
23  from cricketshot.images import load_image
24  from cricketshot.core import normalize
25
26
27  def main():
28      out = ROOT / "data/images/processed"
29      initial = out / "initial_hash_audit"
30      initial.mkdir(exist_ok=True)
31      for name in ("manifest.csv", "audit.json", "features.npz"):
32          if not (initial / name).exists():
33              shutil.copy2(out / name, initial / name)
34      evidence = ROOT / "results/images_initial_hash"
35      evidence.mkdir(exist_ok=True)
36      for name in (
37          "metrics.json",
38          "test_predictions.npz",
39          "test_predictions.csv",
40          "latency.json",
41      ):
42          p = ROOT / "results/images" / name
43          if p.exists() and not (evidence / name).exists():
44              shutil.copy2(p, evidence / name)
45      rows = list(
46          csv.DictReader((initial / "manifest.csv").open(encoding="utf-8", newline=""))
47      )
48      with np.load(initial / "features.npz", allow_pickle=False) as a:
49          data = {k: a[k] for k in a.files}
50      audit = json.loads((initial / "audit.json").read_text())
51      parent = list(range(len(rows)))
52
53      def find(i):
54          while parent[i] != i:
55              parent[i] = parent[parent[i]]
56              i = parent[i]
57          return i
58
59      def union(i, j):
60          parent[find(j)] = find(i)
61
62      old_groups = {}
63      for i, row in enumerate(rows):
64          if row["group"] in old_groups:
65              union(i, old_groups[row["group"]])
66          else:
67              old_groups[row["group"]] = i
68      # ORB descriptors are matched only as an audit tool, never as label features.
69      orb = cv2.ORB_create(nfeatures=400, edgeThreshold=5, patchSize=15, fastThreshold=7)
70      hashes = []
71      thumbs = []
72      keypoints = []
73      descriptors = []
```

scripts/strengthen_image_groups.py

```
    areas = []
    for i, row in enumerate(rows):
        im = load_image(ROOT / row["path"])
        im.thumbnail((384, 384))
        variants = []
        for rot in (0, 90, 180, 270):
            variant = im.rotate(rot, expand=True)
            variants.extend([variant, ImageOps.mirror(variant)])
        hashes.append([int(str(imagehash.phash(v)), 16) for v in variants])
        thumbs.append(
            [np.asarray(v.resize((16, 16)), dtype=np.int16) for v in variants]
        )
        gray = cv2.cvtColor(np.asarray(im), cv2.COLOR_RGB2GRAY)
        kp, desc = orb.detectAndCompute(gray, None)
        keypoints.append(np.array([k.pt for k in kp], dtype=np.float32))
        descriptors.append(desc)
        areas.append(im.width * im.height)
        if (i + 1) % 1000 == 0:
            print("Transform audit descriptors", i + 1, flush=True)
    rotation_edges = []
    for i in range(len(rows)):
        for j in range(i):
            if find(i) == find(j):
                continue
            # Transform i into the orientation of j. Eight variants cover quarter
            # turns and mirrors; arbitrary rotations rely on local-feature checks.
            for k, h in enumerate(hashes[i]):
                if (h ^ hashes[j][0]).bit_count() <= 6 and np.abs(
                    thumbs[i][k] - thumbs[j][0]
                ).mean() <= 24:
                    union(i, j)
                    rotation_edges.append((i, j))
                    break
    print("New transform hash edges", len(rotation_edges), flush=True)
    x = normalize(data["features"][:, 0, :])
    candidates = set()
    with threadpool_limits(limits=2):
        nn = NearestNeighbors(
            n_neighbors=16, metric="cosine", algorithm="brute", n_jobs=2
        ).fit(x)
        distances, neighbors = nn.kneighbors(x)
    for i, (dd, ii) in enumerate(zip(distances, neighbors)):
        for distance, j in zip(dd[1:], ii[1:]):
            if distance <= 0.20 and find(i) != find(int(j)):
                candidates.add(tuple(sorted((i, int(j)))))
    matcher = cv2.BFMatcher(cv2.NORM_HAMMING)
    orb_edges = []
    for n, (i, j) in enumerate(sorted(candidates)):
        if find(i) == find(j):
            continue
        d1, d2 = descriptors[i], descriptors[j]
        if d1 is None or d2 is None or min(len(d1), len(d2)) < 12:
            continue
        pairs = matcher.knnMatch(d1, d2, k=2)
        good = [
            pair[0]
            for pair in pairs
            if len(pair) == 2 and pair[0].distance < 0.72 * pair[1].distance
        ]
        if len(good) < 16:
            continue
        p1 = np.float32([keypoints[i][m.queryIdx] for m in good])
        p2 = np.float32([keypoints[j][m.trainIdx] for m in good])
        h, mask = cv2.findHomography(
```

scripts/strengthen_image_groups.py

```
            p1, p2, cv2.RANSAC, 3.0, maxIters=500, confidence=0.99
        )
        if h is None or mask is None:
            continue
        mask = mask.ravel().astype(bool)
        count = int(mask.sum())
        if count < 12 or mask.mean() < 0.65:
            continue
        cov1 = cv2.contourArea(cv2.convexHull(p1[mask])) / areas[i]
        cov2 = cv2.contourArea(cv2.convexHull(p2[mask])) / areas[j]
        if min(cov1, cov2) < 0.08:
            continue
        union(i, j)
        orb_edges.append(
            {
                "a": rows[i]["id"],
                "b": rows[j]["id"],
                "inliers": count,
                "ratio": float(mask.mean()),
                "coverage": [float(cov1), float(cov2)],
            }
        )
        if len(orb_edges) % 100 == 0:
            print("Verified geometric edges", len(orb_edges), flush=True)
components = {}
for i in range(len(rows)):
    components.setdefault(find(i), []).append(i)
conflicts = [
    ii for ii in components.values() if len({rows[i]["label"] for i in ii}) > 1
]
excluded = {i for ii in conflicts for i in ii}
kept = [i for i in range(len(rows)) if i not in excluded]
final_rows = [rows[i] for i in kept]
labels = data["labels"][kept]
groups = np.array([find(i) for i in kept])
folds = list(
    StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=604).split(
        np.zeros(len(kept)), labels, groups
    )
)
split = np.full(len(kept), "train", dtype="<U5")
split[folds[0][1]] = "test"
split[folds[1][1]] = "val"
from collections import Counter

for i, row in enumerate(final_rows):
    row["group"] = f"photo_{groups[i]:05d}"
    row["split"] = str(split[i])
with (out / "manifest.csv").open("w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=list(final_rows[0]))
    writer.writeheader()
    writer.writerows(final_rows)
final_data = {k: v[kept] for k, v in data.items()}
final_data["groups"] = groups
final_data["split"] = split
np.savez_compressed(out / "features.npz", **final_data)
audit.update(
    {
        "initial_visual_groups": audit["visual_groups"],
        "visual_groups": len(set(groups)),
        "exact_representatives_kept": len(kept),
        "class_counts": dict(Counter(labels)),
        "transform_hash_edges": len(rotation_edges),
        "orb_verified_edges": len(orb_edges),
```

scripts/strengthen_image_groups.py

```
            "orb_candidates": len(candidates),
            "final_conflicting_components": len(conflicts),
            "final_conflicting_images_excluded": len(excluded),
            "split_counts": {
                s: dict(Counter(labels[split == s])) for s in ("train", "val", "test")
            },
            "split_groups": {
                s: len(set(groups[split == s])) for s in ("train", "val", "test")
            },
            "rule": "Original pHash/RGB components + quarter-turn/mirror pHash <=6 and MAD <=24 + ORB/RANSAC verified frozen-feature neighbors",
            "protocol_status": "Strengthened after initial image score and visual QA. Final comparison is exploratory under revised grouping.",
            "warning": "Transform and geometric grouping reduce further leakage but do not establish match or player independence.",
        }
    )
    (out / "audit.json").write_text(json.dumps(audit, indent=2), encoding="utf-8")
    edge_record = {
        "transform_edges": [[rows[i]["id"], rows[j]["id"]] for i, j in rotation_edges],
        "orb_edges": orb_edges,
        "excluded_ids": [rows[i]["id"] for i in sorted(excluded)],
    }
    (out / "transformed_duplicate_edges.json").write_text(
        json.dumps(edge_record, indent=2), encoding="utf-8"
    )
    print(json.dumps(audit, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

scripts/train_evaluate.py

```
"""Fit classifier heads, freeze validation choices and save measured test evidence."""

from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import hashlib
import json
import platform
import time
import warnings
import numpy as np
import sklearn
from PIL import Image, ImageEnhance, ImageFilter
from scipy.optimize import minimize_scalar
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    log_loss,
)
from threadpoolctl import threadpool_limits
from cricketshot.core import Encoder, decode_video, descriptor, softmax

```

scripts/train_evaluate.py

```
30
31  def ece(y, p):
32      confidence, correct = p.max(axis=1), p.argmax(axis=1) == y
33      total = 0.0
34      for lo, hi in zip(np.linspace(0, 1, 11)[:-1], np.linspace(0, 1, 11)[1:]):
35          mask = (confidence > lo) & (confidence <= hi)
36          if mask.any():
37              total += mask.mean() * abs(correct[mask].mean() - confidence[mask].mean())
38      return float(total)
39
40
41  def metrics(y, p, classes):
42      pred = p.argmax(axis=1)
43      return {
44          "n": len(y),
45          "accuracy": float(accuracy_score(y, pred)),
46          "balanced_accuracy": float(balanced_accuracy_score(y, pred)),
47          "macro_f1": float(
48              f1_score(
49                  y,
50                  pred,
51                  labels=np.arange(len(classes)),
52                  average="macro",
53                  zero_division=0,
54              )
55          ),
56          "nll": float(log_loss(y, p, labels=np.arange(len(classes)))),
57          "ece_10_bins": ece(y, p),
58          "brier_multiclass": float(
59              np.mean(np.sum((p - np.eye(len(classes))[y]) ** 2, axis=1))
60          ),
61          "confusion": confusion_matrix(y, pred, labels=np.arange(len(classes))).tolist(),
62          "per_class": classification_report(
63              y,
64              pred,
65              labels=np.arange(len(classes)),
66              target_names=classes,
67              output_dict=True,
68              zero_division=0,
69          ),
70      }
71
72
73  def source_bootstrap(y, p, groups, classes, repeats=2000):
74      rng = np.random.default_rng(17)
75      unique = np.unique(groups)
76      values = []
77      for _ in range(repeats):
78          drawn = rng.choice(unique, size=len(unique), replace=True)
79          idx = np.concatenate([np.flatnonzero(groups == g) for g in drawn])
80          values.append(
81              [
82                  np.mean(p[idx].argmax(1) == y[idx]),
83                  f1_score(
84                      y[idx],
85                      p[idx].argmax(1),
86                      labels=np.arange(len(classes)),
87                      average="macro",
88                      zero_division=0,
89                  ),
90              ]
91          )
92      low, high = np.percentile(values, [2.5, 97.5], axis=0)
93      return {
```

scripts/train_evaluate.py

```
        "method": "Percentile cluster bootstrap over held-out source groups, 2000 draws, seed 17",
        "groups": len(unique),
        "accuracy_95": [float(low[0]), float(high[0])],
        "macro_f1_95": [float(low[1]), float(high[1])],
        "caution": "Few source groups. Intervals describe this test set and omit training variability.",
    }


def select_threshold(y, p):
    choices = []
    for threshold in np.arange(0.30, 0.951, 0.05):
        keep = p.max(axis=1) >= threshold
        if keep.sum() >= max(10, int(np.ceil(0.20 * len(y)))):
            acc = float(np.mean(p[keep].argmax(axis=1) == y[keep]))
            if acc >= 0.90:
                choices.append(
                    (float(threshold), float(keep.mean()), acc, int(keep.sum()))
                )
    if choices:
        th, coverage, acc, n = choices[0]
        return th, {
            "target_met": True,
            "validation_coverage": coverage,
            "validation_accuracy": acc,
            "validation_accepted": n,
        }
    th = 0.80
    keep = p.max(axis=1) >= th
    return th, {
        "target_met": False,
        "reason": "No candidate met 90% accuracy with at least 20% coverage and 10 clips. Fallback 0.80.",
        "validation_coverage": float(keep.mean()),
        "validation_accepted": int(keep.sum()),
        "validation_accuracy": (
            float(np.mean(p[keep].argmax(1) == y[keep])) if keep.any() else None
        ),
    }


def main():
    import argparse

    parser = argparse.ArgumentParser()
    parser.add_argument("--view", choices=["full", "batter_roi"], default="batter_roi")
    parser.add_argument("--result-dir", default="results")
    parser.add_argument("--model-dir", default="models")
    args = parser.parse_args()
    cfg = json.loads((ROOT / "configs/experiment.json").read_text())
    cfg["view"] = args.view
    result_dir = ROOT / args.result_dir
    result_dir.mkdir(exist_ok=True, parents=True)
    model_dir = ROOT / args.model_dir
    model_dir.mkdir(exist_ok=True, parents=True)
    data_path = ROOT / (
        "data/processed/features_fullframe.npz"
        if args.view == "full"
        and (ROOT / "data/processed/features_fullframe.npz").exists()
        else "data/processed/features.npz"
    )
    with np.load(data_path, allow_pickle=False) as a:
        features, colors = a["features"], a["color"]
        labels, split, groups, ids = a["labels"], a["split"], a["groups"], a["ids"]
    classes = sorted(set(labels.tolist()))
    y = np.array([classes.index(s) for s in labels])
```

scripts/train_evaluate.py

```
    tr, va, te = [split == s for s in ("train", "val", "test")]
    test_y = y[te]
    results = {
        "classes": classes,
        "seed": 17,
        "configuration": cfg,
        "models": {},
        "environment": {
            "python": platform.python_version(),
            "sklearn": sklearn.__version__,
            "platform": platform.platform(),
            "processor": platform.processor(),
        },
        "feature_file_sha256": hashlib.sha256(data_path.read_bytes()).hexdigest(),
    }
    predictions = {}
    coefficients = {}
    with threadpool_limits(limits=2):
        for mode in ["color"] + cfg["descriptors"]:
            t = time.perf_counter()
            z = colors if mode == "color" else descriptor(features, mode)
            trials, heads = [], []
            for c in cfg["C_candidates"]:
                head = LogisticRegression(
                    C=c, solver="lbfgs", class_weight="balanced", max_iter=2000
                )
                with warnings.catch_warnings(record=True) as caught:
                    warnings.simplefilter("always")
                    head.fit(z[tr], y[tr])
                val_f1 = f1_score(
                    y[va],
                    head.predict(z[va]),
                    labels=np.arange(len(classes)),
                    average="macro",
                    zero_division=0,
                )
                trials.append(
                    {
                        "C": c,
                        "validation_macro_f1": float(val_f1),
                        "iterations": int(head.n_iter_.max()),
                        "warnings": [str(w.message) for w in caught],
                    }
                )
                heads.append(head)
            best = int(np.argmax([r["validation_macro_f1"] for r in trials]))
            head = heads[best]
            val_logits = head.decision_function(z[va])
            fit = minimize_scalar(
                lambda lt: log_loss(
                    y[va],
                    softmax(val_logits, np.exp(lt)),
                    labels=np.arange(len(classes)),
                ),
                bounds=np.log(cfg["temperature_bounds"]),
                method="bounded",
            )
            temp = float(np.exp(fit.x))
            val_p = softmax(val_logits, temp)
            test_logits = head.decision_function(z[te])
            uncalibrated, calibrated = softmax(test_logits), softmax(test_logits, temp)
            threshold, threshold_info = select_threshold(y[va], val_p)
            accepted = calibrated.max(1) >= threshold
            info = {
```

scripts/train_evaluate.py

```
                "C": cfg["C_candidates"][best],
                "dimension": z.shape[1],
                "temperature": temp,
                "trials": trials,
                "validation": metrics(y[va], val_p, classes),
                "test": metrics(test_y, calibrated, classes),
                "test_uncalibrated": metrics(test_y, uncalibrated, classes),
                "review_threshold": threshold,
                "threshold_selection": threshold_info,
                "test_selective": {
                    "coverage": float(accepted.mean()),
                    "accepted": int(accepted.sum()),
                    "accuracy": (
                        float(
                            np.mean(calibrated[accepted].argmax(1) == test_y[accepted])
                        )
                        if accepted.any()
                        else None
                    ),
                },
                "fit_and_evaluate_seconds": time.perf_counter() - t,
            }
            if mode != "color":
                reversed_p = softmax(
                    descriptor(features[te, ::-1], mode) @ head.coef_.T
                    + head.intercept_,
                    temp,
                )
                info["temporal_reversal"] = metrics(test_y, reversed_p, classes)
                info["reversal_max_probability_change"] = float(
                    np.max(np.abs(reversed_p - calibrated))
                )
            results["models"][mode] = info
            predictions[mode] = calibrated
            coefficients[mode] = (head.coef_, head.intercept_)
            print(
                mode,
                "C",
                info["C"],
                "val F1",
                round(info["validation"]["macro_f1"], 4),
                "test accuracy",
                round(info["test"]["accuracy"], 4),
                "test F1",
                round(info["test"]["macro_f1"], 4),
                flush=True,
            )
    majority = np.bincount(y[tr]).argmax()
    results["majority_baseline"] = {
        "class": classes[majority],
        "accuracy": float(np.mean(test_y == majority)),
        "macro_f1": float(
            f1_score(
                test_y,
                np.full(len(test_y), majority),
                labels=np.arange(len(classes)),
                average="macro",
                zero_division=0,
            )
        ),
    }
    mode = cfg["deployment_descriptor"]
    main_model = results["models"][mode]
    main_model["bootstrap"] = source_bootstrap(
```

scripts/train_evaluate.py

```
286          test_y, predictions[mode], groups[te], classes
287      )
288      coef, intercept = coefficients[mode]
289      np.savez_compressed(model_dir / "classifier.npz", coef=coef, intercept=intercept)
290      model_config = {
291          "mode": mode,
292          "view": cfg.get("view", "batter_roi"),
293          "classes": classes,
294          "temperature": main_model["temperature"],
295          "review_threshold": main_model["review_threshold"],
296          "C": main_model["C"],
297          "frames": 9,
298          "train_clips": int(tr.sum()),
299          "validation_clips": int(va.sum()),
300          "test_clips": int(te.sum()),
301          "threshold_selection": main_model["threshold_selection"],
302          "notes": "Review threshold is a validation heuristic, not a certified error bound or open-set
     detector.",
303      }
304      (model_dir / "classifier.json").write_text(
305          json.dumps(model_config, indent=2), encoding="utf-8"
306      )
307      # Corrupt test frames without fitting any parameter on the test set.
308      rows = list(
309          csv.DictReader(
310              (ROOT / "data/processed/manifest.csv").open(encoding="utf-8", newline="")
311          )
312      )
313      test_rows = [r for r in rows if r["split"] == "test"]
314      encoder = Encoder(view=args.view)
315      corrupt_features = {"dark": [], "blur": []}
316      for i, row in enumerate(test_rows):
317          frames, _ = decode_video(ROOT / row["path"])
318          for name in corrupt_features:
319              transformed = [
320                  (
321                      np.asarray(ImageEnhance.Brightness(Image.fromarray(f)).enhance(0.6))
322                      if name == "dark"
323                      else np.asarray(
324                          Image.fromarray(f).filter(ImageFilter.GaussianBlur(radius=2))
325                      )
326                  )
327                  for f in frames
328              ]
329              corrupt_features[name].append(encoder.encode(transformed))
330          if (i + 1) % 25 == 0:
331              print(f"Robustness {i+1}/{len(test_rows)}", flush=True)
332      results["robustness"] = {}
333      for corruption, feats in corrupt_features.items():
334          results["robustness"][corruption] = {}
335          for name in ("middle", "mean", "phase_delta"):
336              c, b = coefficients[name]
337              pp = softmax(
338                  descriptor(np.stack(feats), name) @ c.T + b,
339                  results["models"][name]["temperature"],
340              )
341              results["robustness"][corruption][name] = metrics(test_y, pp, classes)
342      np.savez_compressed(
343          result_dir / "test_predictions.npz",
344          y=test_y,
345          ids=ids[te],
346          groups=groups[te],
347          **{f"p_{k}": v for k, v in predictions.items()},
348      )
```

scripts/train_evaluate.py

```
    with (result_dir / "test_predictions.csv").open(
        "w", encoding="utf-8", newline=""
    ) as f:
        w = csv.writer(f)
        w.writerow(
            ["id", "true", "predicted", "confidence", "accepted"]
            + [f"p_{c}" for c in classes]
        )
        for i, row in enumerate(test_rows):
            pp = predictions[mode][i]
            w.writerow(
                [
                    row["id"],
                    row["label"],
                    classes[pp.argmax()],
                    float(pp.max()),
                    bool(pp.max() >= main_model["review_threshold"]),
                ]
                + pp.tolist()
            )
    (result_dir / "metrics.json").write_text(
        json.dumps(results, indent=2), encoding="utf-8"
    )
    print("Saved measured evidence to results/metrics.json", flush=True)


if __name__ == "__main__":
    main()
```

scripts/train_fusion.py

```
"""Validation-first learned fusion follow-up; keep original models unchanged.

Run --stage validation first. It freezes the selected gate before --stage test.
Only training groups produce the out-of-fold examples used to fit the gate.
"""

from pathlib import Path
import argparse
import hashlib
import json
import sys
import warnings
import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
from cricketshot.core import softmax
from cricketshot.images import image_descriptor
from cricketshot.fusion import gate_features, fit_gate, mix


def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def write(path, data):
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, indent=2), encoding="utf-8")


def main():
    parser = argparse.ArgumentParser(description=__doc__)
```

scripts/train_fusion.py

```
    parser.add_argument("--stage", choices=("validation", "test"), required=True)
    args = parser.parse_args()
    cfg_path = ROOT / "configs/fusion_experiment.json"
    cfg = json.loads(cfg_path.read_text())
    out = ROOT / "results/fusion"
    out.mkdir(exist_ok=True)
    features_path = ROOT / "data/images/processed/features.npz"
    heads_path = ROOT / "models/image/heads.npz"
    binding = {"config_sha256": sha(cfg_path), "features_sha256": sha(features_path),
               "heads_sha256": sha(heads_path), "script_sha256": sha(Path(__file__)),
               "implementation_sha256": sha(ROOT / "src/cricketshot/fusion.py")}
    with np.load(features_path, allow_pickle=False) as data:
        features, labels, split, groups, ids = [data[k] for k in ("features", "labels", "split", "groups",
"ids")]
    classes = sorted(set(labels.tolist()))
    y = np.array([classes.index(label) for label in labels])
    train, val, test = [np.flatnonzero(split == part) for part in ("train", "val", "test")]
    for a, b in ((train, val), (train, test), (val, test)):
        assert not set(groups[a]) & set(groups[b]), "Outer group leakage"
    with np.load(heads_path, allow_pickle=False) as data:
        heads = {key: data[key] for key in data.files}
    experts = cfg["experts"]
    # No validation-fitted temperatures enter the training out-of-fold scores.
    def expert_scores(indices):
        return np.stack([
            softmax(image_descriptor(features[indices], mode) @ heads[mode + "_coef"].T + heads[mode +
"_intercept"])
            for mode in experts
        ], axis=1)

    if args.stage == "validation":
        from sklearn.linear_model import LogisticRegression
        from sklearn.model_selection import StratifiedGroupKFold
        from sklearn.metrics import f1_score, log_loss
        from threadpoolctl import threadpool_limits

        # Written before fitting. Refuse silently replacing a prior experiment.
        if (out / "selection.json").exists():
            raise RuntimeError("Selection already frozen; preserve it rather than overwrite")
        protocol = {"binding": binding, "config": cfg, "classes": classes,
                    "counts": {"train": len(train), "validation": len(val), "test": len(test)},
                    "test_previously_seen": True}
        write(out / "protocol.json", protocol)
        folds = StratifiedGroupKFold(n_splits=cfg["training_folds"], shuffle=True, random_state=cfg["seed"])
        oof = np.full((len(train), 3, 4), np.nan)
        fold_ids = np.full(len(train), -1, dtype=int)
        fold_audit = []
        with threadpool_limits(limits=2):
            for fold, (fit, held) in enumerate(folds.split(train, y[train], groups[train])):
                assert not set(groups[train[fit]]) & set(groups[train[held]])
                audit = {"fold": fold, "fit_groups": sorted(set(map(int, groups[train[fit]]))),
                         "held_groups": sorted(set(map(int, groups[train[held]]))), "warnings": []}
                for e, mode in enumerate(experts):
                    z = image_descriptor(features[train], mode)
                    head = LogisticRegression(C=cfg["expert_C"], class_weight="balanced", solver="lbfgs",
max_iter=2000)
                    with warnings.catch_warnings(record=True) as caught:
                        warnings.simplefilter("always")
                        head.fit(z[fit], y[train[fit]])
                    audit["warnings"].extend(str(w.message) for w in caught)
                    if caught:
                        raise RuntimeError("An expert fit emitted a warning; inspect before continuing")
                    oof[held, e] = head.predict_proba(z[held])
                fold_ids[held] = fold
```

scripts/train_fusion.py

```
                fold_audit.append(audit)
                print(f"Training fold {fold + 1}/5: {len(held)} held-out images", flush=True)
            assert np.isfinite(oof).all() and (fold_ids >= 0).all()
            raw = gate_features(oof, features[train])
            mean, scale = raw.mean(0), np.maximum(raw.std(0), 0.05)
            x = (raw - mean) / scale
            vp = expert_scores(val)
            vx = (gate_features(vp, features[val]) - mean) / scale
            constant_slopes, constant_bias, constant_fit = fit_gate(np.empty((len(train), 0)), oof, y[train], 0)
            constant, constant_weights = mix(vp, np.empty((len(val), 0)), constant_slopes, constant_bias)
            def scores(p):
                return {"macro_f1": float(f1_score(y[val], p.argmax(1), average="macro")),
                        "accuracy": float((p.argmax(1) == y[val]).mean()),
                        "nll": float(log_loss(y[val], p, labels=np.arange(4)))}
            trials, gates = [], []
            for l2 in cfg["gate_l2_candidates"]:
                slopes, bias, fit_info = fit_gate(x, oof, y[train], l2)
                p, _ = mix(vp, vx, slopes, bias)
                trials.append({"l2": l2, **scores(p), **fit_info})
                gates.append((slopes, bias))
            best = max(range(len(trials)), key=lambda k: (trials[k]["macro_f1"], trials[k]["l2"]))
        old = json.loads((ROOT / "results/images/metrics.json").read_text())
        controls = {"uniform_mean": scores(vp.mean(1)), "constant_weights": scores(constant)}
        strongest = max([m["validation"]["macro_f1"] for m in old["models"].values()] + [m["macro_f1"] for m in
    controls.values()])
        selected = {"binding": binding, "selected_l2": trials[best]["l2"], "trials": trials,
                    "controls": controls, "constant_weights": constant_weights[0].tolist(),
                    "constant_fit": constant_fit, "selected_validation": trials[best],
                    "best_reference_validation_f1": strongest,
                    "deploy_adaptive": trials[best]["macro_f1"] > strongest,
                    "selection_uses_test": False}
        np.savez_compressed(out / "oof_predictions.npz", probabilities=oof, y=y[train], ids=ids[train],
    groups=groups[train], fold=fold_ids)
        write(out / "folds.json", fold_audit)
        model_dir = ROOT / "models/fusion"
        model_dir.mkdir(exist_ok=True)
        slopes, bias = gates[best]
        np.savez_compressed(model_dir / "gate.npz", mean=mean, scale=scale, slopes=slopes, intercept=bias,
                            constant_intercept=constant_bias)
        selected["gate_sha256"] = sha(model_dir / "gate.npz")
        write(out / "selection.json", selected)
        print(json.dumps(selected, indent=2), flush=True)
    else:
        from train_evaluate import metrics
        selected = json.loads((out / "selection.json").read_text())
        assert selected["binding"] == binding, "Frozen selection inputs changed"
        gate_path = ROOT / "models/fusion/gate.npz"
        assert sha(gate_path) == selected["gate_sha256"], "Selected gate changed"
        if (out / "metrics.json").exists():
            raise RuntimeError("Test already evaluated; preserve the recorded result")
        with np.load(gate_path, allow_pickle=False) as data:
            gate = {key: data[key] for key in data.files}
        p = expert_scores(test)
        x = (gate_features(p, features[test]) - gate["mean"]) / gate["scale"]
        adaptive, weights = mix(p, x, gate["slopes"], gate["intercept"])
        constant, _ = mix(p, np.empty((len(test), 0)), np.empty((0, 3)), gate["constant_intercept"])
        predictions = {"uniform_mean": p.mean(1), "constant_weights": constant, "adaptive": adaptive}
        # Paired visual-group bootstrap, fixed 2000 draws; descriptive uncertainty.
        with np.load(ROOT / "results/images/test_predictions.npz", allow_pickle=False) as old:
            assert np.array_equal(ids[test], old["ids"]) and np.array_equal(y[test], old["y"])
            reference = {mode: old["p_" + mode] for mode in ("concat", "contrast")}
        correct = {name: values.argmax(1) == y[test] for name, values in {**reference, **predictions}.items()}
        unique = np.unique(groups[test])
        cluster_n = np.array([(groups[test] == g).sum() for g in unique])
```

scripts/train_fusion.py

```
        rng = np.random.default_rng(17)
        draws = rng.integers(len(unique), size=(2000, len(unique)))
        paired = {}
        for name in ("concat", "contrast", "uniform_mean", "constant_weights"):
            delta = correct["adaptive"].astype(int) - correct[name].astype(int)
            cluster_delta = np.array([delta[groups[test] == g].sum() for g in unique])
            boot = cluster_delta[draws].sum(1) / cluster_n[draws].sum(1)
            paired[name] = {"accuracy_difference": float(delta.mean()), "accuracy_difference_95":
np.quantile(boot, [0.025, 0.975]).tolist(),
                            "gained": int((delta == 1).sum()), "lost": int((delta == -1).sum())}
        evidence = {"classes": classes, "selection_sha256": sha(out / "selection.json"), "binding": binding,
                    "test": {name: metrics(y[test], values, classes) for name, values in predictions.items()},
                    "paired_adaptive_minus": paired, "mean_test_weights": weights.mean(0).tolist(),
                    "caution": cfg["caution"] + " Bootstrap intervals condition on fitted models and this split;
 visual groups are not match identities."}
        np.savez_compressed(out / "test_predictions.npz", y=y[test], ids=ids[test], groups=groups[test],
weights=weights,
                            **{"p_" + name: values for name, values in predictions.items()})
        write(out / "metrics.json", evidence)
        print(json.dumps(evidence, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

scripts/train_hog.py

```
"""One exploratory HOG comparison on the unchanged Option B image split."""
from pathlib import Path
import sys
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
import csv
import hashlib
import json
import time
import warnings
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from threadpoolctl import threadpool_limits
from cricketshot.hog import PARAMETERS, hog_descriptor
from cricketshot.images import load_image
from cricketshot.core import softmax
from train_evaluate import metrics


def file_hash(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()


def main():
    start = time.perf_counter()
    out = ROOT / "results/hog"
    out.mkdir(parents=True, exist_ok=True)
    folder = ROOT / "models/hog"
    folder.mkdir(parents=True, exist_ok=True)
    config = json.loads((ROOT / "configs/hog_experiment.json").read_text())
    manifest = ROOT / "data/images/processed/manifest.csv"
    rows = list(csv.DictReader(manifest.open(encoding="utf-8", newline="")))
    metadata = {"parameters": PARAMETERS, "manifest_sha256": file_hash(manifest),
                "implementation_sha256": file_hash(ROOT / "src/cricketshot/hog.py"),
                "protocol_sha256": file_hash(ROOT / "configs/hog_experiment.json")}
```

scripts/train_hog.py

```
    # Preserve the intended protocol before extracting features or fitting heads.
    (out / "protocol_before_fit.json").write_text(json.dumps({**config, **metadata}, indent=2))
    cache = ROOT / "data/images/processed/hog_features.npz"
    cache_meta = cache.with_suffix(".json")
    if cache.exists() and cache_meta.exists() and json.loads(cache_meta.read_text()) == metadata:
        with np.load(cache, allow_pickle=False) as arrays:
            x = arrays["features"].copy()
            assert arrays["ids"].tolist() == [r["id"] for r in rows]
    else:
        features = []
        for i, row in enumerate(rows, 1):
            features.append(hog_descriptor(load_image(ROOT / row["path"])))
            if i % 500 == 0:
                print(f"HOG features {i}/{len(rows)}", flush=True)
        x = np.stack(features)
        np.savez_compressed(cache, features=x, ids=np.array([r["id"] for r in rows]))
        cache_meta.write_text(json.dumps(metadata, indent=2))
    classes = sorted({r["label"] for r in rows})
    y = np.array([classes.index(r["label"]) for r in rows])
    masks = {s: np.array([r["split"] == s for r in rows]) for s in ("train", "val", "test")}
    trials = []
    heads = []
    with threadpool_limits(limits=2):
        for c in config["C_candidates"]:
            head = LogisticRegression(C=c, class_weight="balanced", solver="lbfgs", max_iter=2000)
            with warnings.catch_warnings(record=True) as caught:
                warnings.simplefilter("always")
                head.fit(x[masks["train"]], y[masks["train"]])
            score = f1_score(y[masks["val"]], head.predict(x[masks["val"]]), average="macro")
            trials.append({"C": c, "validation_macro_f1": float(score),
                           "iterations": int(head.n_iter_.max()),
                           "warnings": [str(w.message) for w in caught]})
            heads.append(head)
            print(f"HOG C={c}, validation macro F1={score:.6f}", flush=True)
        best = int(np.argmax([t["validation_macro_f1"] for t in trials]))
        head = heads[best]
        # Test is scored only after the validation choice is frozen.
        (out / "selection_before_test.json").write_text(json.dumps({"trials": trials, "selected_C":
trials[best]["C"]}, indent=2))
        probabilities = {s: softmax(head.decision_function(x[masks[s]])) for s in ("val", "test")}
    np.savez_compressed(folder / "head.npz", coef=head.coef_, intercept=head.intercept_)
    model_config = {"classes": classes, "parameters": PARAMETERS, "C": trials[best]["C"],
                    "metadata": metadata, "temperature": 1.0, "exploratory": True}
    (folder / "classifier.json").write_text(json.dumps(model_config, indent=2))
    result = {"classes": classes, "config": config, "metadata": metadata, "trials": trials,
              "C": trials[best]["C"], "dimension": x.shape[1],
              "validation": metrics(y[masks["val"]], probabilities["val"], classes),
              "test": metrics(y[masks["test"]], probabilities["test"], classes),
              "elapsed_seconds": time.perf_counter() - start}
    (out / "metrics.json").write_text(json.dumps(result, indent=2))
    test_rows = [r for r in rows if r["split"] == "test"]
    p = probabilities["test"]
    np.savez_compressed(out / "test_predictions.npz", y=y[masks["test"]], p=p,
                        ids=np.array([r["id"] for r in test_rows]),
                        groups=np.array([r["group"] for r in test_rows]))
    with (out / "test_predictions.csv").open("w", encoding="utf-8", newline="") as stream:
        writer = csv.writer(stream)
        writer.writerow(["id", "group", "true", "predicted"] + ["p_" + c for c in classes])
        for row, probability in zip(test_rows, p):
            writer.writerow([row["id"], row["group"], row["label"], classes[probability.argmax()]] +
probability.tolist())
    print(json.dumps({"C": result["C"], "accuracy": result["test"]["accuracy"],
                      "macro_f1": result["test"]["macro_f1"]}), flush=True)

```

scripts/train_hog.py

```
 99
100 if __name__ == "__main__":
101     main()
```

scripts/train_images.py

```
  1 """Fit four image heads under the documented, revised visual-grouping protocol."""
  2
  3 from pathlib import Path
  4 import sys
  5
  6 ROOT = Path(__file__).resolve().parents[1]
  7 sys.path.insert(0, str(ROOT / "src"))
  8 import csv
  9 import json
 10 import time
 11 import warnings
 12 import numpy as np
 13 from PIL import Image, ImageEnhance, ImageFilter, ImageOps
 14 from scipy.optimize import minimize_scalar
 15 from sklearn.linear_model import LogisticRegression
 16 from sklearn.metrics import f1_score, log_loss
 17 from threadpoolctl import threadpool_limits
 18 from cricketshot.core import Encoder, softmax
 19 from cricketshot.images import load_image, image_views, image_descriptor
 20 from train_evaluate import metrics, source_bootstrap
 21
 22
 23 def main():
 24     cfg = json.loads((ROOT / "configs/image_experiment.json").read_text())
 25     out = ROOT / "results/images"
 26     out.mkdir(parents=True, exist_ok=True)
 27     model_dir = ROOT / "models/image"
 28     model_dir.mkdir(exist_ok=True)
 29     with np.load(ROOT / "data/images/processed/features.npz", allow_pickle=False) as a:
 30         features, labels, split, groups, ids = [
 31             a[k] for k in ("features", "labels", "split", "groups", "ids")
 32         ]
 33     classes = sorted(set(labels.tolist()))
 34     y = np.array([classes.index(c) for c in labels])
 35     tr, va, te = [split == s for s in ("train", "val", "test")]
 36     for s in ("train", "val", "test"):
 37         if set(labels[split == s]) != set(classes):
 38             raise ValueError("Class absent from " + s)
 39     results = {
 40         "classes": classes,
 41         "config": cfg,
 42         "models": {},
 43         "n": {s: int((split == s).sum()) for s in ("train", "val", "test")},
 44     }
 45     heads = {}
 46     preds = {}
 47     val_predictions = {}
 48     temperatures = {}
 49     with threadpool_limits(limits=2):
 50         for mode in cfg["modes"]:
 51             z = image_descriptor(features, mode)
 52             trials = []
 53             fitted = []
 54             for c in cfg["C_candidates"]:
 55                 head = LogisticRegression(
 56                     C=c, class_weight="balanced", solver="lbfgs", max_iter=2000
 57                 )
```

scripts/train_images.py

```
                with warnings.catch_warnings(record=True) as caught:
                    warnings.simplefilter("always")
                    head.fit(z[tr], y[tr])
                score = f1_score(y[va], head.predict(z[va]), average="macro")
                trials.append(
                    {
                        "C": c,
                        "validation_macro_f1": float(score),
                        "iterations": int(head.n_iter_.max()),
                        "warnings": [str(w.message) for w in caught],
                    }
                )
                fitted.append(head)
            best = int(np.argmax([t["validation_macro_f1"] for t in trials]))
            head = fitted[best]
            vl = head.decision_function(z[va])
            tl = head.decision_function(z[te])
            fit = minimize_scalar(
                lambda t: log_loss(y[va], softmax(vl, np.exp(t)), labels=np.arange(4)),
                bounds=np.log([0.1, 10]),
                method="bounded",
            )
            temp = float(np.exp(fit.x))
            p = softmax(tl, temp)
            v = softmax(vl, temp)
            heads[mode + "_coef"] = head.coef_
            heads[mode + "_intercept"] = head.intercept_
            preds[mode] = p
            val_predictions[mode] = v
            temperatures[mode] = temp
            results["models"][mode] = {
                "C": cfg["C_candidates"][best],
                "dimension": z.shape[1],
                "temperature": temp,
                "trials": trials,
                "validation": metrics(y[va], v, classes),
                "test": metrics(y[te], p, classes),
                "test_uncalibrated": metrics(y[te], softmax(tl), classes),
            }
            print(
                mode,
                "val F1",
                round(results["models"][mode]["validation"]["macro_f1"], 4),
                "test accuracy",
                round(results["models"][mode]["test"]["accuracy"], 4),
                "test F1",
                round(results["models"][mode]["test"]["macro_f1"], 4),
                flush=True,
            )
    vp = val_predictions["contrast"]
    agree_val = (
        val_predictions["global"].argmax(1) == val_predictions["focus"].argmax(1)
    ) & (val_predictions["global"].argmax(1) == vp.argmax(1))
    options = []
    for threshold in np.arange(0.5, 0.951, 0.05):
        keep = (vp.max(1) >= threshold) & agree_val
        if (
            keep.sum() >= max(20, np.ceil(0.2 * len(vp)))
            and np.mean(vp[keep].argmax(1) == y[va][keep]) >= 0.9
        ):
            options.append(float(threshold))
    threshold = options[0] if options else 0.8
    p = preds["contrast"]
    agreement = (preds["global"].argmax(1) == preds["focus"].argmax(1)) & (
```

scripts/train_images.py

```
            preds["global"].argmax(1) == p.argmax(1)
        )
        keep = (p.max(1) >= threshold) & agreement
        conf_only = p.max(1) >= threshold
        results["review"] = {
            "threshold": threshold,
            "target_met": bool(options),
            "validation_accepted": int(((vp.max(1) >= threshold) & agree_val).sum()),
            "test_accepted": int(keep.sum()),
            "test_coverage": float(keep.mean()),
            "test_accuracy": (
                float(np.mean(p[keep].argmax(1) == y[te][keep])) if keep.any() else None
            ),
            "agreement_fraction": float(agreement.mean()),
            "confidence_only_coverage": float(conf_only.mean()),
            "confidence_only_accuracy": (
                float(np.mean(p[conf_only].argmax(1) == y[te][conf_only]))
                if conf_only.any()
                else None
            ),
        }
        results["models"]["contrast"]["bootstrap"] = source_bootstrap(
            y[te], p, groups[te], classes
        )
        results["models"]["contrast"]["bootstrap"][
            "method"
        ] = "Percentile cluster bootstrap over visual duplicate groups, 2000 draws, seed 17"
        results["models"]["contrast"]["bootstrap"][
            "caution"
        ] = "Visual groups are not matches. Intervals omit training/split variability and may be optimistic if
    related frames remain across splits."
        majority = np.bincount(y[tr]).argmax()
        results["majority"] = {
            "class": classes[majority],
            "accuracy": float(np.mean(y[te] == majority)),
            "macro_f1": float(
                f1_score(
                    y[te],
                    np.full(te.sum(), majority),
                    labels=np.arange(4),
                    average="macro",
                    zero_division=0,
                )
            ),
        }
        np.savez_compressed(model_dir / "heads.npz", **heads)
        config = {
            "classes": classes,
            "mode": "contrast",
            "temperatures": temperatures,
            "review_threshold": threshold,
            "review_rule": "confidence >= threshold AND global/focus/fused head top labels all agree",
            "review_target_met": bool(options),
            "crop_bounds": [0.15, 0.05, 0.85, 0.95],
            "note": "No guarantee of unknown-class rejection or match-independent accuracy.",
        }
        (model_dir / "classifier.json").write_text(
            json.dumps(config, indent=2), encoding="utf-8"
        )
        np.savez_compressed(
            out / "test_predictions.npz",
            y=y[te],
            ids=ids[te],
            groups=groups[te],
```

scripts/train_images.py

```
        agreement=agreement,
        accepted=keep,
        **{"p_" + k: v for k, v in preds.items()}
    )
    rows = list(
        csv.DictReader(
            (ROOT / "data/images/processed/manifest.csv").open(
                encoding="utf-8", newline=""
            )
        )
    )
    test_rows = [r for r in rows if r["split"] == "test"]
    encoder = Encoder(view="full")
    stress = {"dark": [], "blur": [], "flip": []}
    for i, row in enumerate(test_rows):
        im = load_image(ROOT / row["path"])
        for name in stress:
            changed = (
                ImageEnhance.Brightness(im).enhance(0.6)
                if name == "dark"
                else (
                    im.filter(ImageFilter.GaussianBlur(2))
                    if name == "blur"
                    else ImageOps.mirror(im)
                )
            )
            stress[name].append(
                encoder.encode([np.asarray(v) for v in image_views(changed)])
            )
        if (i + 1) % 200 == 0:
            print("Image stress tests", i + 1, "/", len(test_rows), flush=True)
    results["stress"] = {}
    for condition, feats in stress.items():
        results["stress"][condition] = {}
        for mode in ("global", "contrast"):
            pp = softmax(
                image_descriptor(np.stack(feats), mode) @ heads[mode + "_coef"].T
                + heads[mode + "_intercept"],
                temperatures[mode],
            )
            results["stress"][condition][mode] = metrics(y[te], pp, classes)
    (out / "metrics.json").write_text(json.dumps(results, indent=2), encoding="utf-8")
    with (out / "test_predictions.csv").open("w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(
            ["id", "true", "predicted", "confidence", "views_agree", "accepted"]
            + ["p_" + c for c in classes]
        )
        for i, row in enumerate(test_rows):
            writer.writerow(
                [
                    row["id"],
                    row["label"],
                    classes[p[i].argmax()],
                    float(p[i].max()),
                    bool(agreement[i]),
                    bool(keep[i]),
                ]
                + p[i].tolist()
            )
    print("Image evidence saved", flush=True)


if __name__ == "__main__":
```

scripts/train_images.py

```
249      main()
```

scripts/verify_revision.py

```
  1  """Current-revision checks; keep earlier verification records explicitly separate."""
  2  from pathlib import Path
  3  import sys
  4  ROOT=Path(__file__).resolve().parents[1]
  5  sys.path.insert(0,str(ROOT/"src"))
  6  import csv
  7  import hashlib
  8  import json
  9  import numpy as np
 10  from cricketshot.core import Encoder
 11  from cricketshot.images import load_image,image_views
 12  from cricketshot.hog import PARAMETERS,HOGRecognizer
 13  from analyze_revision import confusion_statistics
 14
 15
 16  def digest(path):return hashlib.sha256(path.read_bytes()).hexdigest()
 17
 18
 19  def main():
 20      out=ROOT/"results/revision";out.mkdir(exist_ok=True)
 21      rows=list(csv.DictReader((ROOT/"data/images/processed/manifest.csv").open(encoding="utf-8",newline="")))
 22      comparison=json.loads((out/"comparison.json").read_text())
 23      original=json.loads((ROOT/"results/images/metrics.json").read_text())
 24      checks=[]
 25      def check(name,condition,detail=""):
 26          checks.append({"name":name,"pass":bool(condition),"detail":detail})
 27          if not condition:raise AssertionError(name+": "+detail)
 28      for name in ("hog","global","focus","concat","contrast"):
 29          path=ROOT/("results/hog/test_predictions.npz" if name=="hog" else "results/images/test_predictions.npz")
 30          with np.load(path,allow_pickle=False) as arrays:
 31              p=arrays["p"] if name=="hog" else arrays["p_"+name]
 32              y=arrays["y"];cm=np.zeros((4,4),dtype=int)
 33              np.add.at(cm,(y,p.argmax(1)),1)
 34          computed=confusion_statistics(cm)
 35          recorded=comparison["models"][name]["test"]
 36          check(name+" independent confusion
    arithmetic",np.allclose(computed,[recorded["accuracy"],recorded["macro_f1"]],atol=1e-12))
 37      with np.load(ROOT/"results/images/test_predictions.npz",allow_pickle=False) as arrays:
 38          accepted=arrays["accepted"];correct=arrays["p_contrast"].argmax(1)==arrays["y"]
 39      check("Review exact counts preserved",int(accepted.sum())==557 and int((accepted&correct).sum())==433)
 40      pair=comparison["paired_contrast_concat"]
 41      check("Paired contrast gain is uncertain",pair["correct_difference"]==3 and
    pair["accuracy_difference_95"][0]<0<pair["accuracy_difference_95"][1])
 42      cal=original["models"]["contrast"]
 43      check("Calibration failure recorded",cal["test"]["ece_10_bins"]>cal["test_uncalibrated"]["ece_10_bins"] and
    cal["test"]["nll"]>cal["test_uncalibrated"]["nll"])
 44      check("Original threshold retained",abs(original["review"]["threshold"]-.70)<1e-12)
 45      model=HOGRecognizer()
 46      check("HOG preprocessing matches model",model.config["parameters"]==PARAMETERS)
 47      meta=json.loads((ROOT/"data/images/processed/hog_features.json").read_text())
 48      check("HOG cache binds manifest and
    implementation",meta["manifest_sha256"]==digest(ROOT/"data/images/processed/manifest.csv") and
    meta["implementation_sha256"]==digest(ROOT/"src/cricketshot/hog.py"))
 49      with np.load(ROOT/"data/images/processed/features.npz",allow_pickle=False) as arrays:
 50          features=arrays["features"].copy()
 51          check("CNN cache IDs match manifest",arrays["ids"].tolist()==[r["id"] for r in rows])
 52      encoder=Encoder(view="full");maximum_error=0.0
 53      for index in np.linspace(0,len(rows)-1,64,dtype=int):
 54          row=rows[index];image=load_image(ROOT/row["path"])
```

scripts/verify_revision.py

```
        pixel_hash=hashlib.sha256(image.width.to_bytes(4,"big")+image.height.to_bytes(4,"big")+image.tobytes()).
    hexdigest()
        check("Raw pixel hash "+row["id"],pixel_hash==row["rgb_sha256"])
        fresh=encoder.encode([np.asarray(v) for v in image_views(image)])
        error=float(np.max(np.abs(fresh-features[index])))
        maximum_error=max(maximum_error,error)
        check("Fresh two-view features "+row["id"],np.allclose(fresh,features[index],atol=1e-6))
    cache_proof={"manifest_sha256":digest(ROOT/"data/images/processed/manifest.csv"),
                 "feature_cache_sha256":digest(ROOT/"data/images/processed/features.npz"),
                 "encoder_sha256":digest(ROOT/"models/mobilenetv2_features.onnx"),
                 "images_module_sha256":digest(ROOT/"src/cricketshot/images.py"),
                 "core_module_sha256":digest(ROOT/"src/cricketshot/core.py"),
                 "fresh_images_checked":64,"two_views_per_image":True,"maximum_absolute_error":maximum_error,
                 "scope":"64 deterministic samples across all splits; not every cached image independently
    recomputed"}
    (out/"cache_validation.json").write_text(json.dumps(cache_proof,indent=2))
    # If the original sibling workspace exists, verify it was not modified.
    sibling=ROOT.parents[1]
    baseline_path=ROOT/".build/local_notes/docs/original_files_baseline.json"
    if baseline_path.exists() and (sibling/"app.py").exists() and (sibling/"revision").exists():
        baseline=json.loads(baseline_path.read_text())
        changed=[item["path"] for item in baseline if not (sibling/item["path"]).exists() or
    digest(sibling/item["path"])!=item["sha256"]]
        check("Original project files preserved",not changed,str(changed))
        preserved=["data/images/processed/manifest.csv","models/image/heads.npz","models/image/classifier.json",

    "results/images/metrics.json","results/images/test_predictions.npz","results/images_initial_hash/metrics.json",

    "results/images_initial_hash/test_predictions.npz","results/metrics.json","results/test_predictions.npz"]
        for name in preserved:check("Preserved evidence "+name,digest(ROOT/name)==digest(sibling/name))
    result={"all_checks_pass":True,"checks":checks,"cache_validation":cache_proof,
            "scope":"Current revision: arithmetic, fixed evidence, preprocessing parity and original
    preservation. Pytest, HTTP, render and document checks have separate records."}
    (out/"verification.json").write_text(json.dumps(result,indent=2))
    print(f"{len(checks)} current revision checks passed; maximum CNN feature discrepancy {maximum_error:g}")


if __name__=="__main__":main()
```

scripts/verify_runtime.py

```
"""Check repository-only inference without training data or development packages.

Uses Flask's request client and an actual multipart upload. With --image, the
input may be a local photograph outside the repository. Otherwise a generated
pattern checks wiring only; no recognition-quality claim follows from it.
"""
from pathlib import Path
import argparse
import hashlib
import io
import json
import sys
import importlib.metadata
from PIL import Image, ImageDraw

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from app import create_app


def main():
    parser = argparse.ArgumentParser(description=__doc__)
```

scripts/verify_runtime.py

```
    parser.add_argument("--image", type=Path)
    parser.add_argument("--output", type=Path)
    args = parser.parse_args()
    client = create_app().test_client()
    home = client.get("/")
    assert home.status_code == 200
    sub = json.loads((ROOT/"configs/submission.json").read_text())
    html = home.get_data(as_text=True)
    assert all(m["name"] in html and m["roll_number"] in html for m in sub["members"])
    assert sub["github_url"] in html
    assert client.get("/video").status_code == 200
    assert client.get("/api/status").json["ready"]
    if args.image:
        payload = args.image.read_bytes()
        filename = args.image.name
    else:
        im = Image.new("RGB", (320, 240), "#448344")
        ImageDraw.Draw(im).line((40, 210, 240, 30), fill="white", width=12)
        stream = io.BytesIO()
        im.save(stream, format="PNG")
        payload, filename = stream.getvalue(), "generated_wiring_check.png"
    input_sha256 = hashlib.sha256(payload).hexdigest()
    response = client.post("/image-api/predict", data={"image": (io.BytesIO(payload), filename)})
    assert response.status_code == 200, response.get_data(as_text=True)
    result = response.json
    for p in (result["probabilities"], result["hog_baseline"]["probabilities"]):
        assert len(p) == 4 and abs(sum(p.values())-1) < 1e-8
    assert len(result["previews"]) == 2
    assert client.post("/image-api/predict", data={"image": (io.BytesIO(b"bad"), "bad.png")}).status_code == 400
    examples = client.get("/image-api/examples").json
    example_metadata = {entry["id"]: entry for entry in
json.loads((ROOT/"demo/image_examples.json").read_text())}
    bundled = [entry for entry in example_metadata.values() if entry.get("bundled")]
    assert len(bundled) == 5 and {e["label"] for e in bundled if e["demo_role"] == "success"} ==
{"drive","legglance-flick","pullshot","sweep"}
    assert {e["id"] for e in bundled}.issubset({e["id"] for e in examples})
    demo_checks=[]
    for available in examples:
        # The public API deliberately omits filesystem paths. Resolve them
        # from the local manifest when checking actual upload bytes.
        entry=example_metadata[available["id"]]
        assert client.get("/image-api/example/"+entry["id"]+"/thumbnail").status_code == 200
        example_payload=(ROOT/entry["path"]).read_bytes()
        assert hashlib.sha256(example_payload).hexdigest()==entry["example_sha256"]
        example=client.post("/image-api/example/"+entry["id"])

upload=client.post("/image-api/predict",data={"image":(io.BytesIO(example_payload),Path(entry["path"]).name)})
        assert example.status_code==upload.status_code==200
        assert example.json["probabilities"]==upload.json["probabilities"]
        assert example.json["hog_baseline"]==upload.json["hog_baseline"]
        assert upload.json["label"]==entry["expected_prediction"]
        if entry["demo_role"]=="success":assert upload.json["label"]==entry["label"]
        if entry["demo_role"]=="confident_error":assert upload.json["label"]!=entry["label"]
        if entry["demo_role"]=="view_disagreement":assert not upload.json["prediction_consistent"]
        demo_checks.append({"id":entry["id"],"role":entry["demo_role"],"bundled":entry["bundled"],"label":upload
.json["label"],"example_upload_parity":True})
    paths = [ROOT/"app.py", ROOT/"configs/submission.json", Path(__file__).resolve()]
    for folder in ("src", "templates", "static", "models"):
        paths.extend(p for p in (ROOT/folder).rglob("*") if p.is_file() and p.suffix in {".py", ".html", ".js",
".css", ".json", ".npz", ".onnx"})
    proof = {"all_pass": True, "scope": "Repository runtime; Flask request client, actual multipart upload,
CNN/HOG probabilities, previews, malformed-upload rejection, team and URL rendering. No new accuracy
measurement.",
             "input": "external photograph" if args.image else "generated wiring pattern",
```

scripts/verify_runtime.py

```
            "input_sha256": input_sha256, "label": result["label"],
            "available_image_examples": len(examples), "example_checks":demo_checks, "python": sys.version,
            "packages": {p: importlib.metadata.version(p) for p in ["numpy", "Pillow", "onnxruntime",
"opencv-python-headless", "Flask"]},
            "source_model_hashes": {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest()
for p in sorted(set(paths))}}
    if args.output:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(json.dumps(proof, indent=2)+"\n", encoding="utf-8")
    print(json.dumps({k: v for k, v in proof.items() if k != "source_model_hashes"}, indent=2))


if __name__ == "__main__":
    main()
```

setup.bat

```
@echo off
cd /d "%~dp0"
python -m venv .venv
if errorlevel 1 exit /b 1
".venv\Scripts\python.exe" -m pip install -r requirements.txt
if errorlevel 1 exit /b 1
echo Setup finished. Run run_demo.bat to open the local demo.
pause
```

src/cricketshot/__init__.py

```
"""Cricket shot recognition with frozen visual features and temporal pooling."""
__version__ = '1.0.0'
```

src/cricketshot/core.py

```
"""Shared decoding, feature extraction, temporal descriptors and safe inference.

Training and the demo import the same functions to avoid preprocessing drift.
The saved classifier is numeric NPZ/JSON, so inference never unpickles objects.
"""

from pathlib import Path
import json
import time
import cv2
import numpy as np
import onnxruntime as ort
from PIL import Image, ImageOps

ROOT = Path(__file__).resolve().parents[2]
N_FRAMES = 9
DISPLAY = {
    "cut": "Cut",
    "defence": "Defence",
    "glance": "Glance",
    "offdrive": "Off drive",
    "ondrive": "On drive",
    "pull": "Pull",
    "sweep": "Sweep",
    "unorthodox": "Unorthodox",
}


```

src/cricketshot/core.py

```
def decode_video(path, max_frames=1500):
    """Decode a trimmed clip in order, then sample nine evenly spaced frames.

    Sequential decoding avoids inaccurate frame seeking on short AVI files.
    Bounds protect the local demo from accidentally processing full matches.
    """
    cap = cv2.VideoCapture(str(path))
    if not cap.isOpened():
        raise ValueError("Cannot open this video. Use an AVI, MP4 or MOV clip.")
    fps = float(cap.get(cv2.CAP_PROP_FPS))
    frames = []
    try:
        while True:
            ok, bgr = cap.read()
            if not ok:
                break
            if len(frames) >= max_frames:
                raise ValueError(
                    "Clip is too long. Please trim it to at most 30 seconds."
                )
            if bgr.shape[0] * bgr.shape[1] > 3840 * 2160:
                raise ValueError("Video exceeds the 4K input limit.")
            rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
            # Retain bounded previews rather than full-resolution video frames.
            image = Image.fromarray(rgb)
            image.thumbnail((640, 480))
            frames.append(np.asarray(image))
    finally:
        cap.release()
    if not frames:
        raise ValueError("The file contains no decodable video frames.")
    duration = len(frames) / fps if fps > 0 else None
    if duration and duration > 30:
        raise ValueError("Clip is too long. Please trim it to at most 30 seconds.")
    indices = np.rint(np.linspace(0, len(frames) - 1, N_FRAMES)).astype(int)
    return [frames[i] for i in indices], {
        "decoded_frames": len(frames),
        "fps": fps,
        "duration_seconds": duration,
        "sample_indices": indices.tolist(),
    }


def crop_frame(rgb, view="batter_roi"):
    """Fixed broadcast-layout crop, not a learned person detector."""
    im = Image.fromarray(np.asarray(rgb, dtype=np.uint8)).convert("RGB")
    if view == "batter_roi":
        w, h = im.size
        im = im.crop(
            (round(0.30 * w), round(0.02 * h), round(0.70 * w), round(0.65 * h))
        )
    elif view != "full":
        raise ValueError("Unknown encoder view")
    return im


def prepare_frame(rgb, view="batter_roi"):
    """Resize with preserved aspect ratio, center crop and ImageNet normalize."""
    im = crop_frame(rgb, view)
    im = ImageOps.fit(im, (224, 224), method=Image.Resampling.BILINEAR)
    a = np.asarray(im, dtype=np.float32) / 255.0
    a = (a - np.array([0.485, 0.456, 0.406], np.float32)) / np.array(
        [0.229, 0.224, 0.225], np.float32
    )
```

src/cricketshot/core.py

```
    return np.ascontiguousarray(a.transpose(2, 0, 1)[None])


def normalize(x):
    """Per-vector L2 normalization; this learns no statistics from any split."""
    return x / np.maximum(np.linalg.norm(x, axis=-1, keepdims=True), 1e-12)


class Encoder:
    def __init__(self, model=None, view="batter_roi"):
        self.view = view
        model = Path(model or ROOT / "models/mobilenetv2_features.onnx")
        if not model.exists():
            raise FileNotFoundError(
                "Missing visual encoder. Run python scripts/prepare_data.py first."
            )
        options = ort.SessionOptions()
        options.intra_op_num_threads = 2
        options.inter_op_num_threads = 1
        self.session = ort.InferenceSession(
            str(model), sess_options=options, providers=["CPUExecutionProvider"]
        )
        self.input_name = self.session.get_inputs()[0].name

    def encode(self, frames):
        """Run frozen ImageNet parameters; no gradient or weight update occurs."""
        embeddings = []
        for frame in frames:
            tensor = prepare_frame(frame, self.view)  # (1, 3, 224, 224)
            output = self.session.run(None, {self.input_name: tensor})[0]
            embeddings.append(output.reshape(-1))  # pooled 1,280-value vector
        return np.stack(embeddings).astype(np.float32)


def descriptor(features, mode):
    """Order-sensitive compact representation. Accepts (..., 9, 1280)."""
    x = normalize(np.asarray(features, np.float32))
    if mode == "middle":
        z = x[..., 4, :]
    elif mode == "mean":
        z = x.mean(axis=-2)
    elif mode in ("phase", "phase_delta"):
        early, middle, late = [x[..., i : i + 3, :].mean(axis=-2) for i in (0, 3, 6)]
        parts = [early, middle, late]
        if mode == "phase_delta":
            parts += [late - early, x.std(axis=-2)]
        z = np.concatenate(parts, axis=-1)
    else:
        raise ValueError(f"Unknown descriptor: {mode}")
    return normalize(z).astype(np.float32)


def color_descriptor(frames):
    """A deliberately shallow baseline for background and color shortcuts."""
    hs = []
    for frame in frames:
        hsv = cv2.cvtColor(frame, cv2.COLOR_RGB2HSV)
        hist = cv2.calcHist([hsv], [0, 1], None, [12, 8], [0, 180, 0, 256]).reshape(-1)
        hs.append(hist / max(hist.sum(), 1))
    return normalize(np.mean(hs, axis=0)).astype(np.float32)


def softmax(logits, temperature=1.0):
    """Convert logits to relative class scores, stably by subtracting the max."""
```

src/cricketshot/core.py

```
    if temperature <= 0:
        raise ValueError("Temperature must be positive")
    z = np.asarray(logits, dtype=np.float64) / temperature
    z -= z.max(axis=-1, keepdims=True)
    p = np.exp(z)
    return p / p.sum(axis=-1, keepdims=True)


class Recognizer:
    def __init__(self, model_dir=None):
        self.model_dir = Path(model_dir or ROOT / "models")
        self.config = json.loads(
            (self.model_dir / "classifier.json").read_text(encoding="utf-8")
        )
        with np.load(self.model_dir / "classifier.npz", allow_pickle=False) as a:
            self.coef, self.intercept = a["coef"].copy(), a["intercept"].copy()
        self.encoder = Encoder(
            self.model_dir / "mobilenetv2_features.onnx",
            view=self.config.get("view", "batter_roi"),
        )

    def predict_features(self, features):
        z = descriptor(features, self.config["mode"])
        logits = z @ self.coef.T + self.intercept
        p = softmax(logits, self.config["temperature"])
        return p

    def predict(self, path):
        t = time.perf_counter()
        frames, meta = decode_video(path)
        features = self.encoder.encode(frames)
        p = self.predict_features(features)
        i = int(p.argmax())
        # This is a confidence-based review rule, not proof that an input is cricket.
        accepted = bool(p[i] >= self.config["review_threshold"])
        return {
            "label": self.config["classes"][i],
            "display_label": DISPLAY.get(
                self.config["classes"][i], self.config["classes"][i]
            ),
            "confidence": float(p[i]),
            "accepted": accepted,
            "decision": "Prediction" if accepted else "Needs review",
            "probabilities": {c: float(v) for c, v in zip(self.config["classes"], p)},
            "threshold": self.config["review_threshold"],
            "mode": self.config["mode"],
            "seconds": time.perf_counter() - t,
            "video": meta,
        }, frames
```

src/cricketshot/fusion.py

```
"""Small, inspectable mixture of frozen view classifiers (no CNN training)."""

import numpy as np
from .core import normalize, softmax


def gate_features(probabilities, embeddings):
    """Scores: (images, experts, classes); embeddings: (images, 2, 1280)."""
    p = np.asarray(probabilities, dtype=np.float64)
    if p.ndim != 3 or p.shape[1:] != (3, 4):
        raise ValueError("Expected three four-class experts")
```

src/cricketshot/fusion.py

```
    if not np.isfinite(p).all() or (p < 0).any() or not np.allclose(p.sum(2), 1):
        raise ValueError("Expert probabilities must be finite and sum to one")
    x = normalize(np.asarray(embeddings, dtype=np.float64))
    if x.ndim != 3 or x.shape[:2] != (len(p), 2):
        raise ValueError("Expected two embeddings per image")
    entropy = -(p * np.log(np.clip(p, 1e-15, 1))).sum(2) / np.log(4)
    ordered = np.sort(p, axis=2)
    margin = ordered[:, :, -1] - ordered[:, :, -2]
    maximum = ordered[:, :, -1]
    distance = np.column_stack([
        0.5 * np.abs(p[:, a] - p[:, b]).sum(1)
        for a, b in ((0, 1), (0, 2), (1, 2))
    ])
    cosine = (1 - (x[:, 0] * x[:, 1]).sum(1))[:, None]
    return np.concatenate([entropy, margin, maximum, distance, cosine], axis=1)


def mix(probabilities, features, slopes, intercept):
    """A convex mixture: weights and output probabilities both sum to one."""
    weights = softmax(np.asarray(features) @ slopes + intercept)
    return (weights[:, :, None] * probabilities).sum(1), weights


def objective(theta, features, probabilities, labels, sample_weights, l2):
    """Balanced NLL and its analytic gradient; shared by fitting and tests."""
    n, dimensions = features.shape
    experts = probabilities.shape[1]
    slopes = theta[:dimensions * experts].reshape(dimensions, experts)
    intercept = theta[dimensions * experts:]
    q, weights = mix(probabilities, features, slopes, intercept)
    true_expert = probabilities[np.arange(n), :, labels]
    true_q = np.clip(q[np.arange(n), labels], 1e-15, 1)
    loss = -(sample_weights * np.log(true_q)).mean() + 0.5 * l2 * (slopes ** 2).sum()
    dz = sample_weights[:, None] * weights * (1 - true_expert / true_q[:, None]) / n
    gradient = np.concatenate([(features.T @ dz + l2 * slopes).ravel(), dz.sum(0)])
    return float(loss), gradient


def fit_gate(features, probabilities, labels, l2):
    """Training only. Constant-weight control passes an (N, 0) feature matrix."""
    from scipy.optimize import minimize

    counts = np.bincount(labels, minlength=probabilities.shape[2])
    if (counts == 0).any():
        raise ValueError("Every class must be present in gate training")
    sample_weights = len(labels) / (len(counts) * counts[labels])
    initial = np.zeros((features.shape[1] + 1) * probabilities.shape[1])
    fitted = minimize(
        objective, initial, args=(features, probabilities, labels, sample_weights, l2),
        jac=True, method="L-BFGS-B", options={"maxiter": 2000, "gtol": 1e-8, "ftol": 1e-12},
    )
    if not fitted.success:
        raise RuntimeError("Gate fit failed: " + str(fitted.message))
    dimensions, experts = features.shape[1], probabilities.shape[1]
    return (
        fitted.x[:dimensions * experts].reshape(dimensions, experts),
        fitted.x[dimensions * experts:],
        {"iterations": int(fitted.nit), "objective": float(fitted.fun), "converged": True},
    )
```

src/cricketshot/hog.py

src/cricketshot/hog.py

```
"""A small, explicit HOG baseline for the Lecture 10 comparison.

This is a documented HOG variant, not a reimplementation claimed to reproduce
Dalal and Triggs exactly. Cells do not interpolate votes across cell boundaries.
Angles interpolate between two unsigned orientation bins. Blocks use L2 norm.
No parameter here is fitted to the dataset.
"""
from pathlib import Path
import json
import numpy as np
from PIL import Image, ImageOps
from .core import ROOT, normalize, softmax
from .images import image_views, load_image

PARAMETERS = {
    "image_size": [224, 224],
    "grayscale": "PIL RGB to L (8-bit luminance)",
    "gradient": "central difference, zero derivative at image boundaries",
    "orientations": 9,
    "angle_range_degrees": [0, 180],
    "pixels_per_cell": [16, 16],
    "cells_per_block": [2, 2],
    "block_stride_cells": 1,
    "orientation_interpolation": "linear between circular neighboring bins",
    "spatial_interpolation": False,
    "block_normalization": "L2, epsilon=1e-5",
    "final_normalization": "L2 per image, epsilon floor=1e-12",
    "dimension": 6084,
}


def prepare_hog(image):
    """Use the same global-view geometry as the CNN; then discard color."""
    global_view, _ = image_views(image)
    square = ImageOps.fit(global_view, (224, 224), method=Image.Resampling.BILINEAR)
    return np.asarray(square.convert("L"), dtype=np.float32) / 255.0


def gradients(gray):
    """Return magnitude and unsigned angle for each grayscale pixel."""
    gx = np.zeros_like(gray)
    gy = np.zeros_like(gray)
    gx[:, 1:-1] = gray[:, 2:] - gray[:, :-2]
    gy[1:-1, :] = gray[2:, :] - gray[:-2, :]
    magnitude = np.hypot(gx, gy)
    angle = np.mod(np.degrees(np.arctan2(gy, gx)), 180.0)
    return magnitude, angle


def cell_histograms(magnitude, angle, cell_size=16, bins=9):
    """Accumulate magnitude, not pixel frequency, into orientation bins.

    For a direction between two bin centers, split its magnitude in proportion
    to distance. The final bin wraps around to zero because angles are unsigned.
    """
    height, width = magnitude.shape
    if height % cell_size or width % cell_size:
        raise ValueError("Image dimensions must be multiples of cell size")
    yy, xx = np.indices((height, width))
    cell_id = (yy // cell_size) * (width // cell_size) + xx // cell_size
    position = angle * (bins / 180.0)
```

src/cricketshot/hog.py

```
    lower = np.floor(position).astype(int) % bins
    fraction = position - np.floor(position)
    upper = (lower + 1) % bins
    count = (height // cell_size) * (width // cell_size) * bins
    low_votes = np.bincount((cell_id * bins + lower).ravel(),
                            weights=(magnitude * (1 - fraction)).ravel(), minlength=count)
    high_votes = np.bincount((cell_id * bins + upper).ravel(),
                             weights=(magnitude * fraction).ravel(), minlength=count)
    return (low_votes + high_votes).reshape(height // cell_size, width // cell_size, bins)


def hog_descriptor(image, return_maps=False):
    """224² -> 14x14 cells -> 13x13 overlapping blocks -> 6,084 values."""
    gray = prepare_hog(image)
    magnitude, angle = gradients(gray)
    cells = cell_histograms(magnitude, angle)
    windows = np.lib.stride_tricks.sliding_window_view(cells, (2, 2), axis=(0, 1))
    # Arrange each block as four neighboring cells, with nine bins per cell.
    blocks = windows.transpose(0, 1, 3, 4, 2).reshape(13, 13, 36)
    blocks = blocks / np.sqrt(np.sum(blocks ** 2, axis=-1, keepdims=True) + 1e-10)
    feature = normalize(blocks.reshape(-1)).astype(np.float32)
    if return_maps:
        return feature, gray, magnitude, cells
    return feature


class HOGRecognizer:
    """Read numeric weights only; share extraction with the training script."""
    def __init__(self, model_dir=None):
        folder = Path(model_dir or ROOT / "models/hog")
        self.config = json.loads((folder / "classifier.json").read_text(encoding="utf-8"))
        if self.config["parameters"] != PARAMETERS:
            raise ValueError("HOG model preprocessing does not match this implementation")
        with np.load(folder / "head.npz", allow_pickle=False) as arrays:
            self.coef = arrays["coef"].copy()
            self.intercept = arrays["intercept"].copy()

    def predict_features(self, feature):
        return softmax(np.asarray(feature) @ self.coef.T + self.intercept)

    def predict(self, path):
        feature = hog_descriptor(load_image(path))
        probabilities = self.predict_features(feature)
        index = int(probabilities.argmax())
        return {"label": self.config["classes"][index],
                "score": float(probabilities[index]),
                "probabilities": dict(zip(self.config["classes"], map(float, probabilities))),
                "note": "Exploratory HOG baseline; uncalibrated softmax scores."}
```

src/cricketshot/images.py

```
"""Four-class still-image recognition with dual-view contrast and review gating."""

from pathlib import Path
import json
import time
import numpy as np
from PIL import Image, ImageOps, UnidentifiedImageError
from .core import Encoder, normalize, softmax, ROOT

LABELS = {
    "drive": "Drive",
    "legglance-flick": "Leg glance / flick",
```

src/cricketshot/images.py

```
    "pullshot": "Pull",
    "sweep": "Sweep",
}


def load_image(path):
    try:
        with Image.open(path) as source:
            if source.width * source.height > 24_000_000:
                raise ValueError("Use an image below 24 megapixels.")
            source.load()
            image = ImageOps.exif_transpose(source).convert("RGB")
    except (UnidentifiedImageError, OSError) as exc:
        raise ValueError(
            "Cannot read this image. Use a valid JPEG, PNG or WebP file."
        ) from exc
    if min(image.size) < 32:
        raise ValueError("Image is too small. Use at least 32 pixels on each side.")
    return image


def image_views(image):
    """Two fixed views. Central crop is a context test, not an actor detector."""
    image = image.copy()
    image.thumbnail((1024, 1024))
    w, h = image.size
    focus = image.crop(
        (round(0.15 * w), round(0.05 * h), round(0.85 * w), round(0.95 * h))
    )
    return image, focus


def image_descriptor(features, mode):
    """Compare the two frozen 1,280-value embeddings without training the CNN.

    Normalize each view first so its vector length cannot dominate fusion.
    The absolute difference adds a non-linear relationship to a linear head.
    It is a regional-view comparison, not a keypoint or bat-motion descriptor.
    """
    x = normalize(np.asarray(features, np.float32))
    if mode == "global":
        z = x[..., 0, :]
    elif mode == "focus":
        z = x[..., 1, :]
    elif mode == "concat":
        z = np.concatenate([x[..., 0, :], x[..., 1, :]], axis=-1)
    elif mode == "contrast":
        z = np.concatenate(
            [x[..., 0, :], x[..., 1, :], np.abs(x[..., 0, :] - x[..., 1, :])], axis=-1
        )
    else:
        raise ValueError("Unknown image descriptor")
    return normalize(z).astype(np.float32)


class ImageRecognizer:
    def __init__(self, model_dir=None):
        self.dir = Path(model_dir or ROOT / "models/image")
        self.config = json.loads(
            (self.dir / "classifier.json").read_text(encoding="utf-8")
        )
        with np.load(self.dir / "heads.npz", allow_pickle=False) as a:
            self.heads = {k: a[k].copy() for k in a.files}
        self.encoder = Encoder(ROOT / "models/mobilenetv2_features.onnx", view="full")
```

src/cricketshot/images.py

```
 77
 78      def predict_features(self, features, mode="contrast"):
 79          """Apply learned cricket weights, then the unchanged validation temperature."""
 80          z = image_descriptor(features, mode)
 81          return softmax(
 82              z @ self.heads[mode + "_coef"].T + self.heads[mode + "_intercept"],
 83              self.config["temperatures"][mode],
 84          )
 85
 86      def predict(self, path):
 87          t = time.perf_counter()
 88          image = load_image(path)
 89          views = image_views(image)
 90          features = self.encoder.encode([np.asarray(v) for v in views])
 91          p = self.predict_features(features)
 92          pg, pf = [self.predict_features(features, m) for m in ("global", "focus")]
 93          agree = bool(pg.argmax() == pf.argmax())
 94          consistent = bool(agree and pg.argmax() == p.argmax())
 95          ci = int(p.argmax())
 96          threshold = self.config["review_threshold"]
 97          accepted = bool(p.max() >= threshold and consistent)
 98          return {
 99              "label": self.config["classes"][ci],
100              "display_label": LABELS[self.config["classes"][ci]],
101              "score_type": "temperature-scaled softmax",
102              "score_note": "A relative score among four supported labels, not a guarantee of correctness. Test
    calibration worsened after temperature scaling.",
103              "confidence": float(p.max()),
104              "accepted": accepted,
105              "decision": "Prediction" if accepted else "Needs review",
106              "probabilities": dict(zip(self.config["classes"], map(float, p))),
107              "views_agree": agree,
108              "prediction_consistent": consistent,
109              "global_label": LABELS[self.config["classes"][int(pg.argmax())]],
110              "focus_label": LABELS[self.config["classes"][int(pf.argmax())]],
111              "threshold": threshold,
112              "seconds": time.perf_counter() - t,
113              "input_size": list(image.size),
114              "scope": "Four-class still-image classification. Unknown classes can receive confident scores.",
115          }, views
```

static/app.js

```
  1 'use strict';
  2 const $ = (id) => document.getElementById(id);
  3 const names = {cut:'Cut',defence:'Defence',glance:'Glance',offdrive:'Off drive',ondrive:'On
    drive',pull:'Pull',sweep:'Sweep',unorthodox:'Unorthodox'};
  4 let selectedFile = null, lastResult = null, busy = false;
  5 function setFile(file) {
  6   selectedFile = file;
  7   $('file-label').textContent = file ? file.name : 'Drop your video here';
  8   $('analyze-button').disabled = !file || busy;
  9 }
 10 $('video-input').addEventListener('change', e => setFile(e.target.files[0]));
 11 $('dropzone').addEventListener('dragover', e => {e.preventDefault(); $('dropzone').classList.add('drag');});
 12 $('dropzone').addEventListener('dragleave', () => $('dropzone').classList.remove('drag'));
 13 $('dropzone').addEventListener('drop', e => {e.preventDefault(); $('dropzone').classList.remove('drag');
    setFile(e.dataTransfer.files[0]);});
 14 function showState(state) {
 15   for (const name of ['empty-state','loading-state','error-state','prediction']) $(name).hidden = name !==
    state;
 16 }
 17 async function analyze(url, options) {
```

static/app.js

```
  18    if (busy) return;
  19    busy = true; $('analyze-button').disabled = true;
  20    document.querySelectorAll('.example').forEach(b => b.disabled = true);
  21    showState('loading-state');
  22    try {
  23      const response = await fetch(url, options);
  24      const data = await response.json();
  25      if (!response.ok) throw new Error(data.error || 'Prediction failed. Please retry.');
  26      lastResult = data; renderResult(data); showState('prediction');
  27    } catch (error) {
  28      $('error-state').textContent = error.message; showState('error-state');
  29    } finally {
  30      busy = false; $('analyze-button').disabled = !selectedFile;
  31      document.querySelectorAll('.example').forEach(b => b.disabled = false);
  32    }
  33  }
  34  $('upload-form').addEventListener('submit', e => {
  35    e.preventDefault(); if (!selectedFile) return;
  36    const body = new FormData(); body.append('video', selectedFile);
  37    analyze('/api/predict', {method:'POST',body});
  38  });
  39  function renderResult(data) {
  40    $('decision').textContent = data.decision.toUpperCase();
  41    $('shot-label').textContent = data.display_label;
  42    $('confidence').textContent = `${(data.confidence*100).toFixed(1)}%`;
  43    $('prediction-context').textContent = data.example ? `Test example: ${names[data.example.label]}.
      ${data.example.note}` : 'Prediction for your uploaded clip';
  44    $('review-note').textContent = data.accepted ? 'Score exceeds the review threshold. Check the clip and the
      alternatives before relying on it.' : 'Score is below the review threshold. The label above is the top candidate
       and needs human review.';
  45    $('review-note').className = `review-note${data.accepted?'':' warn'}`;
  46    const scores = $('scores'); scores.replaceChildren();
  47    for (const [label,p] of Object.entries(data.probabilities).sort((a,b)=>b[1]-a[1])) {
  48      const row=document.createElement('div'); row.className=`class-row${label===data.label?' top':''}`;
  49      const line=document.createElement('div'); line.className='class-label';
  50      const a=document.createElement('span'); a.textContent=names[label]||label;
  51      const b=document.createElement('span'); b.textContent=`${(p*100).toFixed(1)}%`;
  52      line.append(a,b);
  53      const track=document.createElement('div'); track.className='class-track';
  54      const fill=document.createElement('div'); fill.className='class-fill'; fill.style.width=`${p*100}%`;
  55      track.append(fill); row.append(line,track); scores.append(row);
  56    }
  57    $('timing').textContent = `${data.seconds.toFixed(2)} s on CPU`;
  58    const strip=$('phase-strip'); strip.replaceChildren();
  59    ['Early','Middle','Late'].forEach((name,phase) => {
  60      const group=document.createElement('div'); group.className='phase';
  61      const imgs=document.createElement('div'); imgs.className='phase-images';
  62      for(let i=0;i<3;i++) {const im=document.createElement('img'); im.src=data.previews[phase*3+i];
      im.alt=`${name} sample ${i+1}`; imgs.append(im);}
  63      const caption=document.createElement('p'); caption.textContent=name; group.append(imgs,caption);
      strip.append(group);
  64    });
  65  }
  66  $('download-result').addEventListener('click', () => {
  67    if(!lastResult) return;
  68    const {previews,...exported}=lastResult;
  69    const url=URL.createObjectURL(new Blob([JSON.stringify(exported,null,2)],{type:'application/json'}));
  70    const a=document.createElement('a'); a.href=url; a.download='cricketshot-prediction.json'; a.click();
      setTimeout(()=>URL.revokeObjectURL(url),1000);
  71  });
  72  async function initialize() {
  73    try {
  74      const [examplesResponse,evidenceResponse]=await
      Promise.all([fetch('/api/examples'),fetch('/api/evidence')]);
```

static/app.js

```
    const examples=await examplesResponse.json();
    for(const e of examples) {
      const button=document.createElement('button'); button.className='example'; button.type='button';
      button.setAttribute('aria-label',`Analyze ${names[e.label]} test example`);
      const img=document.createElement('img'); img.src=`/api/example/${e.id}/thumbnail`;
img.alt=`${names[e.label]} sample`;
      const caption=document.createElement('span'); caption.textContent=names[e.label];
button.append(img,caption);
      button.addEventListener('click',()=> {if(busy)return;
document.querySelectorAll('.example').forEach(b=>b.classList.remove('selected'));
button.classList.add('selected'); analyze(`/api/example/${e.id}`,{method:'POST'});});
      $('examples').append(button);
    }
    if(evidenceResponse.ok) {
      const evidence=await evidenceResponse.json(), m=evidence.models.phase_delta;
      const items=[[`${(m.test.accuracy*100).toFixed(1)}%`,'Test
accuracy'],[`${(m.test.macro_f1*100).toFixed(1)}%`,'Macro F1'],[String(m.test.n),'Test clips']];
      for(const [value,label] of items) {const box=document.createElement('div');box.className='metric';const
n=document.createElement('strong');n.textContent=value;const
l=document.createElement('span');l.textContent=label;box.append(n,l);$('metrics').append(box);}
    }
  }catch(error){$('examples').textContent='Examples unavailable. Check the setup instructions.';}
}
initialize();
```

static/images.css

```
.header-right
a{color:#4c7042;text-underline-offset:4px;margin-right:20px}.image-views{display:grid;grid-template-columns:1fr
1fr;gap:16px;margin-top:12px}.model-view{display:flex;align-items:center;gap:10px}.model-view
img{height:100px;width:100px;object-fit:contain;background:#edf1e7}.model-view
p{font-size:10px;line-height:1.6;color:#77866e}.model-view
strong{display:block;font-size:11px;color:#3c633a;font-weight:500}.prediction-heading
h3{font-size:30px}.examples{grid-template-columns:repeat(4,1fr)}.example img{height:63px}.example
span{font-size:8px}.scores{margin:20px 0}.hero-fact{margin-right:0;padding-left:35px}.hero-fact
p{font-size:12px}@media(max-width:480px){.model-view{display:block}.model-view
img{width:100%;height:110px}.header-right a{margin-right:3px}.header-right{gap:5px}}
.focus-demo header,.focus-demo .hero,.focus-demo .evidence,.focus-demo .method,.focus-demo
footer{display:none}.focus-demo main{padding:28px;max-width:none}.focus-demo
.workspace{min-height:600px}.focus-demo .model-view img{width:140px;height:140px}
.input-section,.model-view{min-width:0}.image-views{grid-template-columns:repeat(2,minmax(0,1fr))}.prediction-he
ading>div:first-child{min-width:0}.score{flex-shrink:0}@media(max-width:850px){.workspace{grid-template-columns:
minmax(0,1fr)}.input-section,.result-section{padding:24px}.focus-demo main{padding:16px}.focus-demo .model-view
img{width:100%;height:120px}.example-heading>span{font-size:8px}.prediction-heading h3{font-size:26px}.score
strong{font-size:31px}}
.examples{grid-template-columns:repeat(3,minmax(0,1fr))}.example{min-width:0}.example
span{padding-bottom:2px;font-size:10px}.example small{display:block;padding:0 4px
7px;color:#526a44;font-size:9px;line-height:1.4}

footer{gap:24px;align-items:flex-start;line-height:1.8}footer>span:first-child{max-width:780px}footer
a{color:inherit}
```

static/images.js

```
'use strict';
const $=id=>document.getElementById(id);
const labels={'drive':'Drive','legglance-flick':'Leg glance / flick','pullshot':'Pull','sweep':'Sweep'};
const roles={success:'Success',confident_error:'Confident error',view_disagreement:'View disagreement'};
let file=null,busy=false,result=null;
function choose(f){file=f;$('file-label').textContent=f?f.name:'Drop a cricket image
here';$('analyze-button').disabled=!f||busy;}
$('image-input').addEventListener('change',e=>choose(e.target.files[0]));
```

static/images.js

```
$('dropzone').addEventListener('dragover',e=>{e.preventDefault();$('dropzone').classList.add('drag');});
$('dropzone').addEventListener('dragleave',()=>$('dropzone').classList.remove('drag'));
$('dropzone').addEventListener('drop',e=>{e.preventDefault();$('dropzone').classList.remove('drag');choose(e.dataTransfer.files[0]);});
function state(active){for(const id of ['empty-state','loading-state','error-state','prediction'])$(id).hidden=id!==active;}
function clearExampleSelection(){document.querySelectorAll('.example').forEach(x=>x.classList.remove('selected'));}
async function predict(url,options){
  if(busy)return;busy=true;$('analyze-button').disabled=true;document.querySelectorAll('.example').forEach(b=>b.disabled=true);state('loading-state');
  try{const response=await fetch(url,options),data=await response.json();if(!response.ok)throw new Error(data.error||'Recognition failed.');result=data;render(data);state('prediction');}
  catch(error){$('error-state').textContent=error.message;state('error-state');}
  finally{busy=false;$('analyze-button').disabled=!file;document.querySelectorAll('.example').forEach(b=>b.disabled=false);}
}
$('image-form').addEventListener('submit',e=>{e.preventDefault();if(!file||busy)return;clearExampleSelection();const body=new FormData();body.append('image',file);predict('/image-api/predict',{method:'POST',body});});
function render(data){
  $('decision').textContent=data.decision.toUpperCase();$('shot-label').textContent=data.display_label;$('confidence').textContent=`${(data.confidence*100).toFixed(1)}%`;
  $('prediction-context').textContent=data.example?`Dataset label: ${labels[data.example.label]}. ${data.example.note}`:'Prediction for your image';
  const reason=!data.prediction_consistent?'The separate heads do not all agree with this prediction. Human review is needed.':data.confidence<data.threshold?'The score is below the review threshold. Human review is needed.':'All heads agree and the score exceeds the review threshold. A confident answer can still be wrong.';
  $('review-note').textContent=reason;$('review-note').className=`review-note${data.accepted?'':' warn'}`;
  const scores=$('scores');scores.replaceChildren();
  for(const [label,p] of Object.entries(data.probabilities).sort((a,b)=>b[1]-a[1])){const row=document.createElement('div');row.className=`class-row${label===data.label?' top':''}`;const line=document.createElement('div');line.className='class-label';const name=document.createElement('span');name.textContent=labels[label];const val=document.createElement('span');val.textContent=`${(p*100).toFixed(1)}%`;line.append(name,val);const track=document.createElement('div');track.className='class-track';const fill=document.createElement('div');fill.className='class-fill';fill.style.width=`${p*100}%`;track.append(fill);row.append(line,track);scores.append(row);}
  const h=data.hog_baseline; $('hog-result').textContent=h?`HOG baseline: ${labels[h.label]} (${(100*h.score).toFixed(1)}% uncalibrated score). This uses grayscale gradient histograms. Neither model score is a guarantee of correctness.`:'';
  $('timing').textContent=`${(data.seconds*1000).toFixed(0)} ms on CPU`;
  const views=$('image-views');views.replaceChildren();
  ['Global view','Central crop'].forEach((name,i)=>{const box=document.createElement('div');box.className='model-view';const image=document.createElement('img');image.src=data.previews[i];image.alt=name;const p=document.createElement('p');const strong=document.createElement('strong');strong.textContent=name;p.append(strong,document.createTextNode(i===0?data.global_label:data.focus_label));box.append(image,p);views.append(box);});
  $('view-agreement').textContent='The global and crop heads are separate diagnostic classifiers. Their agreement controls the review decision. Displayed pixels match the model crops before normalization.';
}
$('download-result').addEventListener('click',()=>{if(!result)return;const {previews,...data}=result;const url=URL.createObjectURL(new Blob([JSON.stringify(data,null,2)],{type:'application/json'}));const a=document.createElement('a');a.href=url;a.download='cricketshot-image-prediction.json';a.click();setTimeout(()=>URL.revokeObjectURL(url),1000);});
async function init(){try{const [er,mr]=await Promise.all([fetch('/image-api/examples'),fetch('/image-api/evidence')]);const examples=await er.json();if(!examples.length)$('examples').textContent='Upload your own image, or restore selected examples using scripts/prepare_demo_examples.py (see README).';for(const e of examples){const b=document.createElement('button');b.className='example';b.type='button';const role=roles[e.demo_role]||'Example';b.setAttribute('aria-label',`Analyze ${labels[e.label]}: ${role}`);const im=document.createElement('img');im.src=`/image-api/example/${e.id}/thumbnail`;im.alt=labels[e.label];const t=document.createElement('span');t.textContent=e.short_label||labels[e.label];const tag=document.createElement('small');tag.textContent=role;b.append(im,t,tag);b.addEventListener('click',()=>{if(busy)return;clearExampleSelection();b.classList.add('selected');predict(`/image-api/example/${e.id}`,{method:'POST'});});$('examples').append
```

static/images.js

```
(b);}if(mr.ok){const r=await mr.json(),m=r.models.contrast.test;for(const [value,label] of
[[`${(m.accuracy*100).toFixed(1)}%`,'Test accuracy'],[`${(m.macro_f1*100).toFixed(1)}%`,'Macro
F1'],[String(m.n),'Test images']]){const box=document.createElement('div');box.className='metric';const
strong=document.createElement('strong');strong.textContent=value;const span=document.createElement('span');span.
textContent=label;box.append(strong,span);$('metrics').append(box);}}const params=new URLSearchParams(location.s
earch),example=params.get('example');if(params.get('focus')==='1')document.body.classList.add('focus-demo');if(e
xamples.some(e=>e.id===example))await predict(`/image-api/example/${encodeURIComponent(example)}`,{method:'POST'
});}catch(error){$('examples').textContent='Examples unavailable. See README.md for setup.';}}
init();
```

static/style.css

```
@font-face{font-family:system;src:local('Segoe
UI')}*{box-sizing:border-box}body{margin:0;background:#f6f7f2;color:#182c25;font-family:system,'Segoe
UI',Arial,sans-serif}header{height:88px;border-bottom:1px solid
#dfe4d9;display:flex;align-items:center;justify-content:space-between;padding:0 5.8%}.wordmark{font-size:25px;fo
nt-weight:750;letter-spacing:-1px;text-decoration:none;color:#182c25;display:flex;align-items:center}.wordmark>s
pan:first-child{font-weight:400}.wordmark-dot{display:block;width:8px;height:8px;background:#c26644;margin:12px
0 0 5px;border-radius:50%}.header-right{display:flex;align-items:center;gap:8px;font-size:12px;color:#5b6c61}.lo
cal-dot{width:6px;height:6px;background:#547c47;border-radius:50%}.course{margin-left:38px;font-size:11px;letter
-spacing:1px}main{max-width:1400px;margin:0 auto;padding:0
5.8%}.hero{display:flex;justify-content:space-between;align-items:center;padding:60px 0
48px}.eyebrow{font-size:10px;letter-spacing:2px;font-weight:700;color:#5e775d;margin:0 0
18px}h1{font-size:64px;letter-spacing:-3px;line-height:1.05;font-weight:550;margin:0 0 24px}h1 em{font-family:Ge
orgia,serif;font-weight:400;color:#60825c}.intro{font-size:15px;line-height:1.75;color:#667368;max-width:470px;m
argin:0}.hero-fact{border-left:1px solid #ced7c9;padding-left:45px;margin-right:30px}.fact-number{font-family:Ge
orgia,serif;font-size:88px;line-height:1}.fact-number
span{font-size:15px;font-family:system,Arial;margin-left:12px;color:#6f7e6b}.hero-fact
p{font-size:13px}.muted{color:#7b877a;line-height:1.7}.workspace{display:grid;grid-template-columns:42%
58%;background:#fff;border:1px solid #dfe4d9;min-height:560px}.input-section{padding:28px 30px;border-right:1px
solid #dfe4d9}.result-section{padding:28px 32px;min-width:0}.section-title{display:flex;align-items:center;gap:1
2px;margin-bottom:25px}.step{font-size:10px;color:#899481;border:1px solid #dbe1d3;border-radius:50%;width:27px;
height:27px;display:grid;place-content:center}h2{font-size:19px;letter-spacing:-.5px;font-weight:600;margin:0}.d
ropzone{display:flex;flex-direction:column;align-items:center;justify-content:center;min-height:170px;border:1px
 dashed #b7c9ad;background:#f9fbf5;cursor:pointer;gap:7px;text-align:center}.dropzone.drag{background:#eaf1df}.u
pload-symbol{color:#79936a;font-size:32px;font-weight:200}.dropzone strong{font-size:13px}.dropzone
span:not(.upload-symbol){font-size:12px;color:#73836d}.dropzone small{font-size:9px;color:#8a9584;margin-top:10p
x}.primary-button{width:100%;background:#244f38;color:#fff;border:0;padding:15px 18px;font:600 12px system,Arial
;cursor:pointer;margin-top:12px;display:flex;justify-content:space-between}.primary-button:disabled{background:#
d5dfcd;cursor:default;color:#65795c}.primary-button:hover:enabled{background:#326d4b}.input-note{font-size:10px;
line-height:1.65;color:#8a9486;margin:12px 0 24px}.example-heading{display:flex;justify-content:space-between;al
ign-items:center;margin-bottom:10px}.example-heading h3{font-size:12px;font-weight:600;margin:0}.example-heading
>span{font-size:9px;color:#88947f}.examples{display:grid;grid-template-columns:repeat(4,1fr);gap:7px}.example{pa
dding:0;background:transparent;border:1px solid
#e0e6da;cursor:pointer;text-align:left;color:#355235}.example:hover,.example.selected{outline:2px solid
#7d9c67;outline-offset:1px}.example img{width:100%;height:55px;object-fit:cover;display:block}.example
span{display:block;padding:6px 4px;font-size:9px}.empty-state{text-align:center;padding:60px 0
40px}.empty-nine{font:100px Georgia;color:#e1e9d8;line-height:1}.empty-state
h3{font-size:17px;font-weight:500;margin:18px 0 10px}.empty-state p,.loading-state
p{font-size:12px;line-height:1.8;color:#99a090}.loading-state{text-align:center;padding:110px 0}.loading-state
h3{font-size:18px;font-weight:500}.spinner{border:2px solid
#e1e8d7;border-top-color:#547d43;border-radius:50%;width:30px;height:30px;margin:0 auto 20px;animation:spin .7s
linear infinite}@keyframes spin{to{transform:rotate(360deg)}}.error-state{padding:20px;background:#fff1e9;color:
#9d4a28;font-size:13px;line-height:1.6}.prediction-heading{display:flex;justify-content:space-between;align-item
s:center}.prediction-heading .eyebrow{margin:0 0 8px}.prediction-heading h3{font-size:35px;font-weight:550;lette
r-spacing:-1px;margin:0}.score{display:flex;flex-direction:column;align-items:end}.score strong{font:38px
Georgia;color:#48733c}.score
span{font-size:9px;color:#939d8b}.prediction-context{font-size:11px;color:#7b8a71;margin:12px 0}.review-note{fon
t-size:11px;line-height:1.6;background:#f0f4e9;padding:10px;color:#607650}.review-note.warn{background:#fff2df;c
olor:#927038}.scores{display:grid;grid-template-columns:1fr 1fr;gap:8px 16px;margin:18px 0}.class-row{font-size:
10px}.class-label{display:flex;justify-content:space-between;margin-bottom:4px;color:#788771}.class-track{height
:3px;background:#edf1e7}.class-fill{height:100%;background:#adc19a}.class-row.top
.class-fill{background:#3a7147}.class-row.top .class-label{color:#2d5c37;font-weight:650}.frames-heading{display
```

static/style.css

```
:flex;justify-content:space-between;align-items:center;border-top:1px solid
#e5eadd;padding-top:15px;margin-top:20px}.frames-heading h4{font-size:12px;margin:0}.frames-heading span{font-si
ze:9px;color:#8a9681}.phase-strip{display:grid;grid-template-columns:repeat(3,1fr);gap:10px;margin-top:12px}.pha
se{min-width:0}.phase-images{display:grid;grid-template-columns:repeat(3,1fr);gap:2px}.phase
img{width:100%;height:48px;object-fit:cover}.phase p{font-size:9px;color:#78896d;margin:6px
0}.frame-note{font-size:9px;line-height:1.7;color:#97a18e}.text-button{background:none;border:0;padding:5px 0;fo
nt-size:10px;color:#557847;cursor:pointer;text-decoration:underline;text-underline-offset:3px}.evidence{display:
flex;justify-content:space-between;gap:30px;margin:42px 0;padding-bottom:32px;border-bottom:1px solid
#dce3d3}.evidence .eyebrow{margin-bottom:12px}.evidence p:not(.eyebrow){font-size:11px;line-height:1.7;color:#87
937e;max-width:340px}.metrics{display:flex;gap:36px;align-items:center}.metric strong{display:block;font:28px
Georgia;color:#325b36;margin-bottom:8px}.metric
span{font-size:10px;color:#87937e;display:block}.method{margin-bottom:40px}.method summary{font-size:12px;cursor
:pointer;display:flex;justify-content:space-between;list-style:none}.method-body{display:grid;grid-template-colu
mns:repeat(3,1fr);gap:28px;font-size:11px;line-height:1.9;color:#75826d;padding-top:15px}footer{border-top:1px
solid #dce3d3;padding:20px 0 30px;display:flex;justify-content:space-between;color:#95a08c;font-size:9px}button:
focus-visible,a:focus-visible,label:focus-within,summary:focus-visible{outline:3px solid #bc784f;outline-offset:
3px}[hidden]{display:none!important}@media(max-width:850px){.hero-fact{display:none}h1{font-size:50px}.workspace
{grid-template-columns:1fr}.input-section{border-right:0;border-bottom:1px solid #dfe4d9}.evidence{flex-directio
n:column}.metrics{justify-content:space-between}.method-body{grid-template-columns:1fr;gap:0}.header-right{font-
size:10px}.course{display:none}.hero{padding-top:38px}footer{gap:20px}.result-section{min-height:450px}}@media(p
refers-reduced-motion:reduce){.spinner{animation:none}}
```

templates/images.html

```
1  <!doctype html>
2  <html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,
   initial-scale=1"><title>CricketShot | Image recognition</title><link rel="stylesheet"
   href="/static/style.css"><link rel="stylesheet" href="/static/images.css"><script src="/static/images.js"
   defer></script></head>
3  <body><header><a class="wordmark" href="/">Cricket<span>Shot</span><span class="wordmark-dot"></span></a><div
   class="header-right"><a href="/video">Video pilot</a><span class="local-dot"></span> Local inference <span
   class="course">EE604 / 2026</span></div></header>
4  <main><div class="hero"><div><p class="eyebrow">CRICKET SHOT RECOGNITION</p><h1>Read the shape<br>of a
   <em>shot.</em></h1><p class="intro">Two views of one image. A lightweight classifier compares the full view with
    a central crop, then shows its prediction and uncertainty.</p></div><div class="hero-fact"><div
   class="fact-number">04<span>shot classes</span></div><p>Drive. Leg glance / flick. Pull. Sweep.</p><p
   class="muted">Real images. Measured results.<br>Every prediction stays on your computer.</p></div></div>
5  <div class="workspace"><section class="input-section"><div class="section-title"><span
   class="step">01</span><h2>Choose an image</h2></div><form id="image-form"><label class="dropzone" id="dropzone"
   for="image-input"><span class="upload-symbol">+</span><strong id="file-label">Drop a cricket image
   here</strong><span>or click to browse</span><small>JPEG, PNG, WebP, BMP / up to 24
   megapixels</small></label><input id="image-input" type="file" accept="image/jpeg,image/png,image/webp,image/bmp"
    hidden><button type="submit" id="analyze-button" class="primary-button" disabled>Recognize shot <span
   aria-hidden="true">&gt;</span></button></form><p class="input-note">Use one visible batter playing a shot. The
   model supports four dataset labels and can be wrong on unfamiliar images.</p><div
   class="example-heading"><h3>Explore held-out examples</h3><span>Includes a failure case</span></div><div
   id="examples" class="examples"></div></section>
6  <section class="result-section" aria-live="polite"><div class="section-title"><span
   class="step">02</span><h2>Inspect the result</h2></div><div id="empty-state" class="empty-state"><div
   class="empty-nine">2</div><h3>One image, two perspectives</h3><p>See class scores, the two input views<br>and
   whether the model requests review.</p></div><div id="loading-state" class="loading-state" hidden><div
   class="spinner"></div><h3>Comparing the views</h3><p>Extracting visual features on CPU...</p></div><div
   id="error-state" class="error-state" role="alert" hidden></div><div id="prediction" hidden><div
   class="prediction-heading"><div><p id="decision" class="eyebrow"></p><h3 id="shot-label"></h3></div><div
   class="score"><strong id="confidence"></strong><span>model score</span></div></div><p id="prediction-context"
   class="prediction-context"></p><p id="review-note" class="review-note"></p><div id="scores"
   class="scores"></div><div class="frames-heading"><h4>Actual model inputs</h4><span id="timing"></span></div><div
    id="image-views" class="image-views"></div><p id="view-agreement" class="frame-note"></p><p id="hog-result"
   class="frame-note"></p><button class="text-button" id="download-result">Download prediction JSON
   </button></div></section></div>
7  <section class="evidence"><div><p class="eyebrow">MEASURED, WITH LIMITS</p><h2>Evidence behind the
   prediction</h2><p>Exact duplicates collapse before splitting. Audited visual groups stay together. The 945 test
   images belong to 70 groups; match and player identities are unavailable.</p></div><div id="metrics"
```

templates/images.html

```
class="metrics"></div></section><details class="method"><summary>Method, scope and limitations
<span>+</span></summary><div class="method-body"><p>A frozen MobileNetV2 extracts two feature vectors. The
classifier combines them with their absolute difference. Only the linear cricket classifiers are trained. The
fixed crop describes a region; it does not detect or segment the batter. A HOG baseline compares grayscale
gradients on the same image split. The contrast feature is a modest course-level extension of established
methods.</p><p>The review rule requires a high enough model score and agreement between the two single-view
heads. It is a secondary diagnostic. Test calibration worsened after temperature scaling (ECE 0.043 to 0.094). A
 model score is not a guarantee of correctness or an unknown-class detector.</p><p>The dataset has no verified
match IDs. Closely related frames may remain across splits despite visual grouping. The separate eight-class
video pilot performs much worse and is available through the link
above.</p></div></details><footer><span>CricketShot / EE604 course project<br><a href="{{ submission.github_url
}}">GitHub repository</a><br>{% for member in submission.members %}{{ member.name }} ({{ member.roll_number
}}){% if not loop.last %} / {% endif %}{% endfor %}</span><span>Local inference. No cloud upload. Predictions
need judgment.</span></footer></main></body></html>
```

templates/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
  <title>CricketShot | EE604</title>
  <link rel="stylesheet" href="/static/style.css">
  <script src="/static/app.js" defer></script>
</head>
<body>
<header><a class="wordmark" href="/">Cricket<span>Shot</span><span class="wordmark-dot"></span></a><div
class="header-right"><span class="local-dot"></span> <a href="/">Main image project</a> <span
class="course">EE604 / 2026</span></div></header>
<main>
  <div class="hero"><div><p class="eyebrow">EXPLORATORY VIDEO BACKUP</p><h1>Every shot has<br>a
<em>sequence.</em></h1><p class="intro">This pilot scores 30.2% accuracy, equal to the majority baseline, and
21.3% macro F1. It is retained as an exploratory result. The main project classifies one image into four
labels.</p></div><div class="hero-fact"><div class="fact-number">09<span>frames</span></div><p>Preparation.
Contact. Follow-through.</p><p class="muted">Three temporal phases, one prediction.<br>Eight classes from real
broadcast clips.</p></div></div>
  <div class="workspace">
    <section class="input-section" aria-labelledby="input-title"><div class="section-title"><span
class="step">01</span><h2 id="input-title">Choose a clip</h2></div>
      <form id="upload-form"><label class="dropzone" id="dropzone" for="video-input"><span
class="upload-symbol">+</span><strong id="file-label">Drop your video here</strong><span>or click to
browse</span><small>AVI, MP4, MOV, MKV, WebM / up to 30 s / 40 MB</small></label><input id="video-input"
type="file" accept="video/*,.avi,.mkv" hidden><button type="submit" id="analyze-button" class="primary-button"
disabled>Analyze clip <span aria-hidden="true">&gt;</span></button></form>
      <p class="input-note">Use one trimmed shot in a typical end-on broadcast view, with the batter near the
upper center. The model uses a fixed crop.</p>
      <div class="example-heading"><h3>Try the test set</h3><span>Ground-truth labels included</span></div><div
id="examples" class="examples"></div>
    </section>
    <section class="result-section" aria-labelledby="result-title" aria-live="polite"><div
class="section-title"><span class="step">02</span><h2 id="result-title">Read the prediction</h2></div>
      <div id="empty-state" class="empty-state"><div class="empty-nine">9</div><h3>A shot unfolds over
time</h3><p>Your prediction, class scores and sampled<br>frames will appear here.</p></div>
      <div id="loading-state" class="loading-state" hidden><div class="spinner"></div><h3>Reading the
sequence</h3><p>Sampling frames and comparing temporal phases...</p></div>
      <div id="error-state" class="error-state" role="alert" hidden></div>
      <div id="prediction" hidden><div class="prediction-heading"><div><p id="decision" class="eyebrow"></p><h3
id="shot-label"></h3></div><div class="score"><strong id="confidence"></strong><span>model
score</span></div></div><p id="prediction-context" class="prediction-context"></p><p id="review-note"
class="review-note"></p><div id="scores" class="scores"></div><div class="frames-heading"><h4>What the model
saw</h4><span id="timing"></span></div><div id="phase-strip" class="phase-strip"></div><p
class="frame-note">Uniform samples divided into early, middle and late phases. These are temporal bins, not
detected contact events.</p><button class="text-button" id="download-result">Download prediction JSON
```

templates/index.html

```
    </button></div>
24      </section>
25    </div>
26    <section class="evidence"><div><p class="eyebrow">MEASURED, WITH LIMITS</p><h2>Evidence behind the
    demo</h2><p>Clips from a source group stay in one split. Test scores below come from saved
    predictions.</p></div><div id="metrics" class="metrics"></div></section>
27    <details class="method"><summary>How it works, and where it can fail <span>+</span></summary><div
    class="method-body"><p>A fixed crop emphasizes the upper center of each broadcast frame. A frozen MobileNetV2
    encoder and a small classifier use ordered phase averages, late-minus-early differences and temporal variation.
    The strip shows these crops.</p><p>A validation-fitted temperature adjusts confidence. Low scores request
    review. This rule cannot reliably detect non-cricket footage or unknown shot classes, and a confident prediction
     can still be wrong.</p><p>Eight labels come from a small, imbalanced archive. The crop revision followed a
    failed full-frame pilot, so the reported scores are exploratory results on a reused holdout. Different camera
    layouts may place the batter outside the crop.</p></div></details>
28    <footer><span>CricketShot / EE604 course project<br><a href="{{ submission.github_url }}">GitHub
    repository</a><br>{% for member in submission.members %}{{ member.name }} ({{ member.roll_number }}){% if not
    loop.last %} / {% endif %}{% endfor %}</span><span>Local inference. No cloud upload. No claim of coaching
    accuracy.</span></footer>
29  </main></body></html>
```

tests/test_fusion.py

```
1   """Check the learned mixture's numerical objective and probability constraints."""
2
3   import numpy as np
4   import csv
5   import hashlib
6   import json
7   from pathlib import Path
8   from scipy.optimize import check_grad
9   from cricketshot.fusion import objective, mix, gate_features, fit_gate
10
11
12  def test_gate_gradient_and_probability_constraints():
13      rng = np.random.default_rng(604)
14      p = rng.dirichlet(np.ones(4), size=(12, 3))
15      x = rng.normal(size=(12, 5))
16      y = np.arange(12) % 4
17      theta = rng.normal(scale=0.1, size=18)
18      args = (x, p, y, np.ones(12), 0.1)
19      error = check_grad(lambda t: objective(t, *args)[0], lambda t: objective(t, *args)[1], theta)
20      assert error < 1e-6
21      q, weights = mix(p, x, theta[:15].reshape(5, 3), theta[15:])
22      np.testing.assert_allclose(q.sum(1), 1)
23      np.testing.assert_allclose(weights.sum(1), 1)
24      assert (q >= 0).all() and (weights >= 0).all()
25
26
27  def test_constant_control_learns_the_reliable_expert():
28      y = np.arange(40) % 4
29      good = np.full((40, 4), 0.05)
30      good[np.arange(40), y] = 0.85
31      bad = np.roll(good, 1, axis=1)
32      p = np.stack([good, bad, np.full_like(good, 0.25)], axis=1)
33      x = np.empty((40, 0))
34      slopes, bias, info = fit_gate(x, p, y, 0)
35      q, weights = mix(p, x, slopes, bias)
36      assert info["converged"] and (weights[:, 0] > 0.99).all()
37      np.testing.assert_array_equal(q.argmax(1), y)
38
39
40  def test_gate_inputs_agreement_and_uniform_scores():
41      p = np.full((2, 3, 4), 0.25)
```

tests/test_fusion.py

```
    embedding = np.ones((2, 2, 8))
    values = gate_features(p, embedding)
    assert values.shape == (2, 13)
    np.testing.assert_allclose(values[:, :3], 1)
    np.testing.assert_allclose(values[:, 3:6], 0)
    np.testing.assert_allclose(values[:, 9:], 0, atol=1e-7)


def test_recorded_folds_exclude_each_predicted_training_group():
    root = Path(__file__).resolve().parents[1]
    rows = list(csv.DictReader((root / "data/images/processed/manifest.csv").open(encoding="utf-8", newline="")))
    expected = {row["id"]: int(row["group"].removeprefix("photo_")) for row in rows if row["split"] == "train"}
    folds = json.loads((root / "results/fusion/folds.json").read_text())
    with np.load(root / "results/fusion/oof_predictions.npz", allow_pickle=False) as data:
        assert set(data["ids"]) == set(expected)
        assert len(set(data["ids"])) == len(data["ids"])
        # sklearn preserves float32 feature precision in these expert outputs.
        np.testing.assert_allclose(data["probabilities"].sum(2), 1, rtol=0, atol=2e-7)
        for row in folds:
            fit, held = set(row["fit_groups"]), set(row["held_groups"])
            assert fit.isdisjoint(held) and fit | held == set(expected.values())
            selected = data["fold"] == row["fold"]
            assert set(data["groups"][selected]) == held
            assert all(expected[str(i)] == int(g) for i, g in zip(data["ids"][selected], data["groups"][selected]))


def test_frozen_selection_and_recorded_metrics():
    from sklearn.metrics import f1_score
    root = Path(__file__).resolve().parents[1]
    selection_path = root / "results/fusion/selection.json"
    selection = json.loads(selection_path.read_text())
    recorded = json.loads((root / "results/fusion/metrics.json").read_text())
    assert not selection["selection_uses_test"]
    assert recorded["selection_sha256"] == hashlib.sha256(selection_path.read_bytes()).hexdigest()
    assert selection["gate_sha256"] == hashlib.sha256((root / "models/fusion/gate.npz").read_bytes()).hexdigest()
    with np.load(root / "results/fusion/test_predictions.npz", allow_pickle=False) as data:
        for mode, metrics in recorded["test"].items():
            predicted = data["p_" + mode].argmax(1)
            assert abs((predicted == data["y"]).mean() - metrics["accuracy"]) < 1e-12
            assert abs(f1_score(data["y"], predicted, average="macro") - metrics["macro_f1"]) < 1e-12
```

tests/test_images.py

```
from pathlib import Path
import csv
import io
import json
import sys
import numpy as np
import pytest
from PIL import Image
ROOT=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(ROOT))
from cricketshot.images import ImageRecognizer,image_descriptor,image_views,load_image
from app import create_app

def test_contrast_descriptor_dimensions():
    x=np.random.default_rng(4).normal(size=(3,2,1280)).astype('float32')
    assert image_descriptor(x,'contrast').shape==(3,3840)
    assert image_descriptor(x,'concat').shape==(3,2560)
```

tests/test_images.py

```
    assert np.allclose(np.linalg.norm(image_descriptor(x,'contrast'),axis=1),1)

def test_image_group_and_hash_disjointness():
    rows=list(csv.DictReader((ROOT/'data/images/processed/manifest.csv').open(encoding='utf-8',newline='')))
    for key in ('group','rgb_sha256'):
        sets={s:{r[key] for r in rows if r['split']==s} for s in ('train','val','test')}
        assert not sets['train']&sets['test'] and not sets['train']&sets['val'] and not sets['val']&sets['test']
    assert len(rows)==len({r['rgb_sha256'] for r in rows})

def test_image_export_and_metrics_parity():
    model=ImageRecognizer();r=json.loads((ROOT/'results/images/metrics.json').read_text())
    with np.load(ROOT/'data/images/processed/features.npz',allow_pickle=False) as a:
        test=a['features'][a['split']=='test']
    with np.load(ROOT/'results/images/test_predictions.npz',allow_pickle=False) as a:
        for mode in ('global','focus','concat','contrast'):
            p=model.predict_features(test,mode)
            assert np.allclose(p,a['p_'+mode],atol=1e-7)
            assert np.allclose(p.sum(1),1)
            assert abs(np.mean(p.argmax(1)==a['y'])-r['models'][mode]['test']['accuracy'])<1e-12
        keep=(a['p_contrast'].max(1)>=model.config['review_threshold'])&a['agreement']
        assert np.array_equal(keep,a['accepted'])

def test_fresh_image_inference_matches_cached():
    rows=list(csv.DictReader((ROOT/'data/images/processed/manifest.csv').open(encoding='utf-8',newline='')))
    row=next(r for r in rows if r['split']=='test')
    result,_=ImageRecognizer().predict(ROOT/row['path'])
    with np.load(ROOT/'results/images/test_predictions.npz',allow_pickle=False) as a:
        i=list(a['ids']).index(row['id']);assert
np.allclose(list(result['probabilities'].values()),a['p_contrast'][i],atol=1e-6)

def test_invalid_images(tmp_path):
    p=tmp_path/'bad.jpg';p.write_bytes(b'not image')
    with pytest.raises(ValueError):load_image(p)
    p=tmp_path/'tiny.png';Image.new('RGB',(3,3)).save(p)
    with pytest.raises(ValueError):load_image(p)

def test_image_api_upload_and_examples():
    c=create_app().test_client();assert c.get('/').status_code==200
    assert c.post('/image-api/predict').status_code==400
    assert c.post('/image-api/predict',data={'image':(io.BytesIO(b'bad'),'bad.jpg')}).status_code==400
    assert c.post('/image-api/predict',data={'image':(io.BytesIO(b'bad'),'bad.svg')}).status_code==400
    e=c.get('/image-api/examples').json;assert len(e)>=3
    assert all(x['split']=='test' and 'path' not in x for x in e)
    p=c.post('/image-api/example/'+e[0]['id']);assert p.status_code==200 and len(p.json['previews'])==2
    assert c.get('/image-api/example/not-real/thumbnail').status_code==404
    # Exercise the real upload endpoint using an actual test example, not a mock.
    sample=json.loads((ROOT/'demo/image_examples.json').read_text())[0]
    with (ROOT/sample['path']).open('rb') as f:
        upload=c.post('/image-api/predict',data={'image':(io.BytesIO(f.read()),'sample.jpg')})
    assert upload.status_code==200
    assert abs(sum(upload.json['probabilities'].values())-1)<1e-8
```

tests/test_project.py

```
"""Tests target leakage, inference parity, malformed inputs and measured evidence."""
from pathlib import Path
import csv
import io
import json
import sys
import numpy as np
import pytest
ROOT=Path(__file__).resolve().parents[1]
```

tests/test_project.py

```
sys.path.insert(0,str(ROOT))
from cricketshot.core import descriptor, softmax, Recognizer, decode_video
from app import create_app

def test_probabilities_stable_and_normalized():
    p=softmax(np.array([[10000,10001,-10000],[0,0,0]]))
    assert np.isfinite(p).all()
    assert np.allclose(p.sum(1),1)
    assert p[0].argmax()==1
    with pytest.raises(ValueError):softmax([1,2],0)

def test_temporal_descriptor_obeys_order():
    x=np.random.default_rng(4).normal(size=(2,9,1280)).astype('float32')
    assert descriptor(x,'phase_delta').shape==(2,6400)
    assert np.allclose(descriptor(x,'mean'),descriptor(x[:,::-1],'mean'),atol=1e-6)
    assert not np.allclose(descriptor(x,'phase_delta'),descriptor(x[:,::-1],'phase_delta'))

@pytest.fixture(scope='module')
def evidence():
    path=ROOT/'data/processed/manifest.csv'
    if not path.exists():pytest.skip('Dataset has not been prepared')
    return list(csv.DictReader(path.open(encoding='utf-8',newline='')))

def test_split_has_no_shared_sources_or_hashes(evidence):
    for key in ('source','group','sha256'):
        sets={s:{r[key] for r in evidence if r['split']==s} for s in ('train','val','test')}
        assert not sets['train'] & sets['test']
        assert not sets['train'] & sets['val']
        assert not sets['val'] & sets['test']
    classes={r['label'] for r in evidence}
    assert len(classes)==8
    for s in ('train','val','test'):
        assert {r['label'] for r in evidence if r['split']==s}==classes

def test_saved_metrics_recompute_from_predictions():
    r=json.loads((ROOT/'results/metrics.json').read_text())
    with np.load(ROOT/'results/test_predictions.npz',allow_pickle=False) as a:
        for mode,info in r['models'].items():
            p=a['p_'+mode]
            assert np.isfinite(p).all() and np.allclose(p.sum(1),1)
            assert abs(np.mean(p.argmax(1)==a['y'])-info['test']['accuracy'])<1e-12
            assert np.sum(info['test']['confusion'])==len(a['y'])
        assert len(np.unique(a['ids']))==len(a['ids'])

def test_numeric_export_matches_training_predictions(evidence):
    model=Recognizer()
    with np.load(ROOT/'data/processed/features.npz',allow_pickle=False) as a:
        pp=model.predict_features(a['features'][a['split']=='test'])
    with np.load(ROOT/'results/test_predictions.npz',allow_pickle=False) as a:
        assert np.allclose(pp,a['p_phase_delta'],atol=1e-7)

def test_live_decode_and_feature_parity(evidence):
    row=next(r for r in evidence if r['split']=='test')
    result,frames=Recognizer().predict(ROOT/row['path'])
    with np.load(ROOT/'results/test_predictions.npz',allow_pickle=False) as a:
        i=list(a['ids']).index(row['id']);expected=a['p_phase_delta'][i]
    assert len(frames)==9
    assert np.allclose(list(result['probabilities'].values()),expected,atol=1e-6)

def test_invalid_video_rejected(tmp_path):
    bad=tmp_path/'bad.avi';bad.write_bytes(b'not a video')
    with pytest.raises(ValueError):decode_video(bad)

def test_local_api_errors_and_templates():
```

tests/test_project.py

```
    client=create_app().test_client()
    assert client.get('/').status_code==200
    assert client.get('/api/status').json['ready']
    assert client.post('/api/predict').status_code==400
    bad=client.post('/api/predict',data={'video':(io.BytesIO(b'wrong'),'bad.exe')})
    assert bad.status_code==400
    invalid=client.post('/api/predict',data={'video':(io.BytesIO(b'wrong'),'bad.mp4')})
    assert invalid.status_code==400
    assert client.post('/api/example/../../app.py').status_code==404
    assert client.post('/api/example/not-real').status_code==404

def test_example_api_runs_real_inference():
    client=create_app().test_client()
    examples=client.get('/api/examples').json
    assert examples
    for e in examples:
        assert e['split']=='test' and 'path' not in e
    response=client.post('/api/example/'+examples[0]['id'])
    assert response.status_code==200
    data=response.json
    assert len(data['previews'])==9
    assert abs(sum(data['probabilities'].values())-1)<1e-8
    assert data['accepted']==(data['confidence']>=data['threshold'])
```

tests/test_revision.py

```
"""Checks for HOG vote arithmetic, cache binding, export and API parity."""
from pathlib import Path
import csv
import io
import json
import hashlib
import pytest
import numpy as np
from PIL import Image
from cricketshot.hog import (PARAMETERS, HOGRecognizer, cell_histograms,
                             gradients, hog_descriptor)
from cricketshot.images import load_image
ROOT = Path(__file__).resolve().parents[1]


def test_hog_votes_are_magnitude_weighted_and_interpolated():
    magnitude = np.array([[90.0]])
    angle = np.array([[50.0]])
    cells = cell_histograms(magnitude, angle, cell_size=1)
    assert np.allclose(cells.sum(), 90)
    # Nine unsigned bins have centers at 0,20,40,...,160 degrees.
    assert cells[0, 0, 2] == 45 and cells[0, 0, 3] == 45
    wrap = cell_histograms(np.array([[20.0]]), np.array([[170.0]]), cell_size=1)
    assert wrap[0, 0, 8] == 10 and wrap[0, 0, 0] == 10


def test_hog_geometry_flat_image_and_unsigned_edges():
    flat = Image.new("RGB", (320, 240), (40, 40, 40))
    x = hog_descriptor(flat)
    assert x.shape == (6084,) and np.isfinite(x).all() and np.count_nonzero(x) == 0
    ramp = np.tile(np.linspace(0, 1, 224), (224, 1)).astype(np.float32)
    m, a = gradients(ramp)
    assert np.allclose(a[:, 1:-1], 0) and np.all(m[:, 1:-1] > 0)
    mr, ar = gradients(1-ramp)
    assert np.allclose(mr, m, atol=1e-6) and np.allclose(ar[:, 1:-1], 0)


```

tests/test_revision.py

```
def test_hog_saved_features_and_export_match_fresh_inputs():
    meta = json.loads((ROOT / "data/images/processed/hog_features.json").read_text())
    assert meta["parameters"] == PARAMETERS
    for key, path in [("manifest_sha256", "data/images/processed/manifest.csv"),
                      ("implementation_sha256", "src/cricketshot/hog.py"),
                      ("protocol_sha256", "configs/hog_experiment.json")]:
        assert meta[key] == hashlib.sha256((ROOT/path).read_bytes()).hexdigest()
    rows = list(csv.DictReader((ROOT / "data/images/processed/manifest.csv").open(encoding="utf-8",
newline="")))
    model = HOGRecognizer()
    with np.load(ROOT / "data/images/processed/hog_features.npz", allow_pickle=False) as cache:
        features = cache["features"]
        assert cache["ids"].tolist() == [r["id"] for r in rows]
        for index in np.linspace(0, len(rows)-1, 8, dtype=int):
            fresh = hog_descriptor(load_image(ROOT / rows[index]["path"]))
            assert np.allclose(fresh, features[index], atol=1e-7)
        test = np.array([r["split"] == "test" for r in rows])
        with np.load(ROOT / "results/hog/test_predictions.npz", allow_pickle=False) as predictions:
            p = model.predict_features(features[test])
            assert np.allclose(p, predictions["p"], atol=1e-7)
            assert np.allclose(p.sum(1), 1)


def test_uploaded_image_compares_both_real_models():
    from app import create_app
    client = create_app().test_client()
    example = next(e for e in json.loads((ROOT / "demo/image_examples.json").read_text()) if e["demo_role"] ==
"confident_error")
    payload = (ROOT / example["path"]).read_bytes()
    response = client.post("/image-api/predict", data={"image": (io.BytesIO(payload), "failure.png")})
    assert response.status_code == 200
    result = response.json
    assert "hog_baseline" in result and result["score_type"] == "temperature-scaled softmax"
    expected = HOGRecognizer().predict(ROOT / example["path"])
    assert result["hog_baseline"]["label"] == expected["label"]
    assert np.allclose(list(result["hog_baseline"]["probabilities"].values()),
list(expected["probabilities"].values()))


def test_bundled_examples_cover_all_labels_and_preserve_demo_behaviour():
    """A public clone must show all classes and the documented real error."""
    from app import create_app
    client = create_app().test_client()
    manifest = json.loads((ROOT / "demo/image_examples.json").read_text())
    bundled = [e for e in manifest if e.get("bundled")]
    assert len(bundled) == 5
    assert {e["label"] for e in bundled if e["demo_role"] == "success"} == {"drive", "legglance-flick",
"pullshot", "sweep"}
    assert sum(e["demo_role"] == "confident_error" for e in bundled) == 1
    for e in bundled:
        assert hashlib.sha256((ROOT/e["path"]).read_bytes()).hexdigest() == e["example_sha256"]
        result = client.post("/image-api/example/"+e["id"])
        assert result.status_code == 200
        assert result.json["label"] == e["expected_prediction"]
        if e["demo_role"] == "success":
            assert result.json["label"] == e["label"]
        else:
            assert result.json["label"] != e["label"] and result.json["confidence"] > .95


def test_posthoc_view_diagnostics_reconcile_with_saved_predictions():
    """Check both decision directions and that agreement partitions the test."""
    d = json.loads((ROOT/"results/revision/view_diagnostics.json").read_text())
    assert d["prediction_archive_sha256"] ==
```

tests/test_revision.py

```
    hashlib.sha256((ROOT/"results/images/test_predictions.npz").read_bytes()).hexdigest()
    with np.load(ROOT/"results/images/test_predictions.npz", allow_pickle=False) as a:
        concat_ok = a["p_concat"].argmax(1) == a["y"]
        contrast_ok = a["p_contrast"].argmax(1) == a["y"]
        assert d["strata"]["all"]["gained"] == int((contrast_ok & ~concat_ok).sum())
        assert d["strata"]["all"]["lost"] == int((concat_ok & ~contrast_ok).sum())
    for key in ("n", "concat_correct", "contrast_correct", "gained", "lost"):
        assert d["strata"]["all"][key] == d["strata"]["views_agree"][key]+d["strata"]["views_disagree"][key]
    all_test = d["strata"]["all"]
    assert all_test["gained"]-all_test["lost"] == all_test["contrast_correct"]-all_test["concat_correct"] == 3


@pytest.mark.parametrize("external", [False, True])
def test_runtime_verifier_cli_exercises_available_examples(tmp_path, external):
    """The initial input hash must survive checks of later demo examples."""
    import subprocess
    import sys
    proof=tmp_path/"runtime.json"
    command=[sys.executable,str(ROOT/"scripts/verify_runtime.py"),"--output",str(proof)]
    if external:
        entry=next(e for e in json.loads((ROOT/"demo/image_examples.json").read_text()) if e.get("bundled"))
        source=ROOT/entry["path"]
        initial_bytes=source.read_bytes()
        command.extend(["--image",str(source)])
    else:
        from PIL import ImageDraw
        pattern=Image.new("RGB", (320,240), "#448344")
        ImageDraw.Draw(pattern).line((40,210,240,30),fill="white",width=12)
        buffer=io.BytesIO();pattern.save(buffer,format="PNG")
        initial_bytes=buffer.getvalue()
    run=subprocess.run(command,
                       cwd=ROOT,capture_output=True,text=True)
    assert run.returncode == 0, run.stdout+run.stderr
    result=json.loads(proof.read_text())
    assert result["all_pass"] and len(result["example_checks"])>=5
    assert sum(e["bundled"] for e in result["example_checks"]) == 5
    assert result["input_sha256"] == hashlib.sha256(initial_bytes).hexdigest()
    assert result["input"] == ("external photograph" if external else "generated wiring pattern")


def test_missing_optional_examples_are_hidden(monkeypatch):
    import app as demo_app
    absent = "demo/images/not_distributed.png"
    original = Path.is_file
    monkeypatch.setattr(Path, "is_file", lambda p: False if p.as_posix().endswith(absent) else original(p))
    original_read = Path.read_text
    def read(p, *args, **kwargs):
        if p.name == "image_examples.json":
            return json.dumps([{"id": "absent", "path": absent}])
        return original_read(p, *args, **kwargs)
    monkeypatch.setattr(Path, "read_text", read)
    client = demo_app.create_app().test_client()
    assert client.get("/image-api/examples").json == []
    assert client.post("/image-api/example/absent").status_code == 404
```